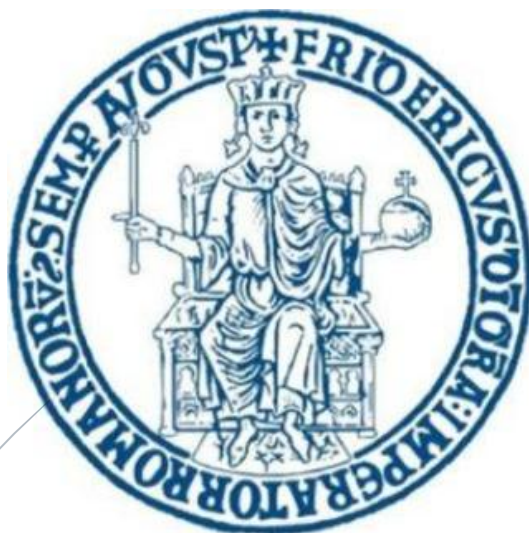


2025/2026 – Team D3

Documentazione di progetto: Software Architecture Design

Interfaccia utente



A.Coppola, A.Pironti, M. Berrino
UNIVERSITA' DEGLI STUDI FEDERICO II

Sommario

Identificazione del Team e definizione degli obiettivi del progetto.	5
CAPITOLO 1 Requisiti e Progettazione.....	6
1.1 Analisi dei Requisiti	6
1.1.1 Obiettivi principali del progetto:	9
1.1.2 Elenco dei requisiti	10
1.2 Progettazione della soluzione	12
1.2.1 Analisi dell'impatto	13
1.2.2 Pattern da seguire	15
1.3 Studio del database in T23	16
1.3.1 Configurazione iniziale del container	16
1.3.2 Accesso al database tramite IntelliJ	17
1.3.3 Connessione tramite plugin database	18
1.3.4 Visualizzazione diagrammi del database	19
1.4 Entità principali	21
1.4.1 Gestione dei progressi del giocatore	21
1.4.2 Sistema di achievement.....	22
1.4.3 Funzionalità social	23
1.4.4 Notifiche	23
1.4.5 Sicurezza e autenticazione.....	24
1.5 Class diagram	25
1.5.1 Il Nucleo dell'identità Player e UserProfile	27
1.5.2 Gamification e Persistenza: PlayerProgress e GameProgress	28
1.5.3 Organizzazione Social.....	29
1.6 Analisi pagina AS-IS	30
1.6.1 Analisi delle criticità dello stato attuale.....	31
1.6.2 Casi d'uso.....	37
CAPITOLO 2 Implementazione	41
2.1 Implementazione e struttura del progetto.....	41
2.1.1 Comunicazione T5-T23	42
2.2 Implementazione di alcuni casi d'uso	43
2.2.1 Logica di controllo: gestione accesso e fall-back	43
2.2.2 Dinamica social: aggiornamento asincrono dello stato	46
2.2.3 Logica di editing profilo: diagramma delle attività	50
2.2.4 Vista dati: modellazione della persistenza per lo storico partite	54

2.3 Descrizione delle decisioni di progetto	57
2.4 Nuova organizzazione degli JavaScript per il profilo	59
2.5 Modifiche apportate ai file	61
2.6 GameProgressService	63
CAPITOLO 3 Testing	65
3.1 Testing e validazione	65
3.1.1 Test dei requisiti funzionali:	65
3.1.2 Test dei requisiti non funzionali:	78
CAPITOLO 4 Sviluppi futuri	79
4.1 Evoluzioni future	79
Riferimenti Bibliografici e Sitografia	80
Bibliografia	80
Sitografia	80

Sommario delle tabelle

Tabella 1 Requisiti funzionali	10
Tabella 2 Requisiti non funzionali	11
Tabella 3 Cambiamenti	14
Tabella 4 Problemi riscontrati	36
Tabella 5 Decisioni da implementare	58
Tabella 6 Modifiche apportate al progetto	62

Sommario delle figure

Figura 1 Diagramma dei componenti del sistema e interazioni tra i moduli 1	7
Figura 2 Diagramma dei componenti del sistema e interazioni tra i moduli 2	7
Figura 3 Component Diagram	12
Figura 4 Blocchi impattanti	13
Figura 5 Pattern architetturale	15
Figura 6 Navigazione del database	18
Figura 7 Diagramma generato automaticamente dal plugin	19
Figura 8 Schema del database - Gestione profilo giocatore	20
Figura 9 Diagramma delle classi	25
Figura 10 Pagina profilo iniziale 1	30
Figura 11 Pagina profilo iniziale 2	30
Figura 12 Endpoint usato in GetUser (T5)	32
Figura 13 Endpoint effettivo in T23	32
Figura 14 Funzione per l'estrazione dell'ID utente del token, restituisce un Long	33
Figura 15 UserId viene passato a GetUser senza essere convertito da Long a String	33
Figura 16 Analisi del grado di accoppiamento tra la logica di presentazione (HTML) e la logica applicativa (JS)	34

Figura 17 Casi d'uso.....	37
Figura 18 Prototipo della vista iniziale profilo	41
Figura 19 Prototipo della vista social	41
Figura 20 Iterazione tra container	42
Figura 21 Inizializzazione del profilo	44
Figura 22 Schermata main con autenticazione.....	45
Figura 23 Pagina profilo main di un giocatore	45
Figura 24 Activity diagram del toggle follow.....	47
Figura 25 Vista elenco dei following	48
Figura 26 Vista dei follow già seguiti	48
Figura 27 Vista follow non seguiti	48
Figura 28 Vista della ricerca dei giocatori.....	49
Figura 29 Activity diagram della modifica e salvataggio profilo	51
Figura 30 Vista di default profilo	52
Figura 31 Modale per la scelta dell'avatar (standard)	52
Figura 32 Schermata modifica della bio.....	53
Figura 33 State machine diagram per l'editing del profilo	53
Figura 34 GameProgressVisibility	55
Figura 35 Vista dello storico partite	56
Figura 36 Microservizio T5	59
Figura 37 T5, resources	60
Figura 38 Ubicazione di GameProgressService.java	63
Figura 39 Funzione di GameProgressService	64
Figura 40 Diagramma del pattern Service-Repository	64
Figura 41 Correttezza caricamento pagina profilo	65
Figura 42 Visualizzazione messaggio di mancato achievements.....	66
Figura 43 Achivements della singola partita	66
Figura 44 Achivements globali	66
Figura 45 Vista obiettivi generici.....	67
Figura 46 Vista degli obiettivi per partita singola.....	67
Figura 47 Avviso errore numero caratteri nickname	68
Figura 48 Avviso errore bio.....	69
Figura 49 Vista profilo modificato.....	69
Figura 50 Modifiche nel DB dopo aver effettuato il salvataggio delle modifiche.....	69
Figura 51 Verifica visualizzazione delle statistiche.....	70
Figura 52 Verifica visualizzazione delle partite	70
Figura 53 Verifica visualizzazione delle partite	71
Figura 54 Punti esperienza presenti nel database.....	72
Figura 55 Verifica nel caso di punti ottenuti superiori al massimo	72
Figura 56 Punti esperienza minori del punteggio massimo	72
Figura 57 Punti minori del massimo	73
Figura 58 Punteggio pari al punteggio massimo.....	73
Figura 59 Punti ottenuti pari al massimo	73
Figura 60 Partite disputate dai giocatori.....	74
Figura 61 Storico delle partite	74
Figura 62 Visualizzazione delle partite giocate	75
Figura 63 Vista iniziale dei seguiti	76

Figura 64 Vista iniziale dei Followers	76
Figura 65 Stato iniziale, giocatore "Marione" è seguito dall'utente.....	76
Figura 66 Rimozione del following "Marione"	77
Figura 67 Vista dei follower successiva al "Smetti di seguire" riferito a "Marione"	77
Figura 68 Stato successivo al "Smetti di seguire" riferito a "Marione".....	77

Identificazione del Team e definizione degli obiettivi del progetto.

Campo	Dettaglio
ID Team	Gruppo D3
Componenti	Andrea Pironti, Antonio Coppola, Mario Berrino
Task Assegnato	Task R6, Gestione profilo giocatore
Approccio	Sviluppo Incrementale, ogni settimana si è scelto un requisito da sviluppare, partendo dalla progettazione, passando per l'integrazione e finendo con il testing end-to-end, per poi passare al prossimo requisito.
Repository Originale	https://github.com/Testing-Game-SAD-2023/A13
Repository Consegnato	https://github.com/antoniocoppola0119-lgtm/A13/tree/main

CAPITOLO 1 Requisiti e Progettazione

1.1 Analisi dei Requisiti

Il task in questione è quello con l'identificativo R6:

- Al momento le informazioni sul profilo del giocatore non sono presentate in maniera organica
- Alcune info sul profilo sono mantenute in T23 e mostrate tramite bottoni del Menù in alto a destra della home page
- Potrebbe essere utile offrire al giocatore la possibilità di accedere ad una pagina dedicata al suo Profilo per mostrare:
 - Immagine e biografia del giocatore
 - Riepilogo proprie Statistiche (Partite giocate, Punti Esperienza, Badge guadagnati...)
 - Classifiche Giocatori
 - Game History
 - Notifiche/ Messaggi
 - **Followers e giocatori seguiti**
 - **Giocatori da poter seguire**
 - ...
-

Task da svolgere – R6 (Profilo Giocatore)

Task	Descrizione	Team
Task R6	Gestire la pagina dedicata al profilo del Giocatore, prevedendo (ad esempio) una parte fissa (Foto/Avatar e Bio del giocatore) ed una parte variabile di informazioni da scegliersi usando un selettore sulla UI (Vedi selettore della pagina Achievement) Spunti: - Il selettore consente di scegliere le informazioni da mostrare volta per volta, evitando di affollare la Pagina Profilo con troppe informazioni - La soluzione si presta a trattare anche esigenze future di visualizzazione di altre informazioni - Valutare se mantenere l'attuale visualizzazione sulla UI degli achievement già disponibile	
Impatto stimato	T5- T23 (rivedere in particolare I Model del Profilo in T23) - valutare se aggiungere un nuovo microservizio di gestione dati del profilo	

Dalla presentazione del task indicato in precedenza, si sono estrapolati i seguenti requisiti e ne sono stati aggiunti altri:

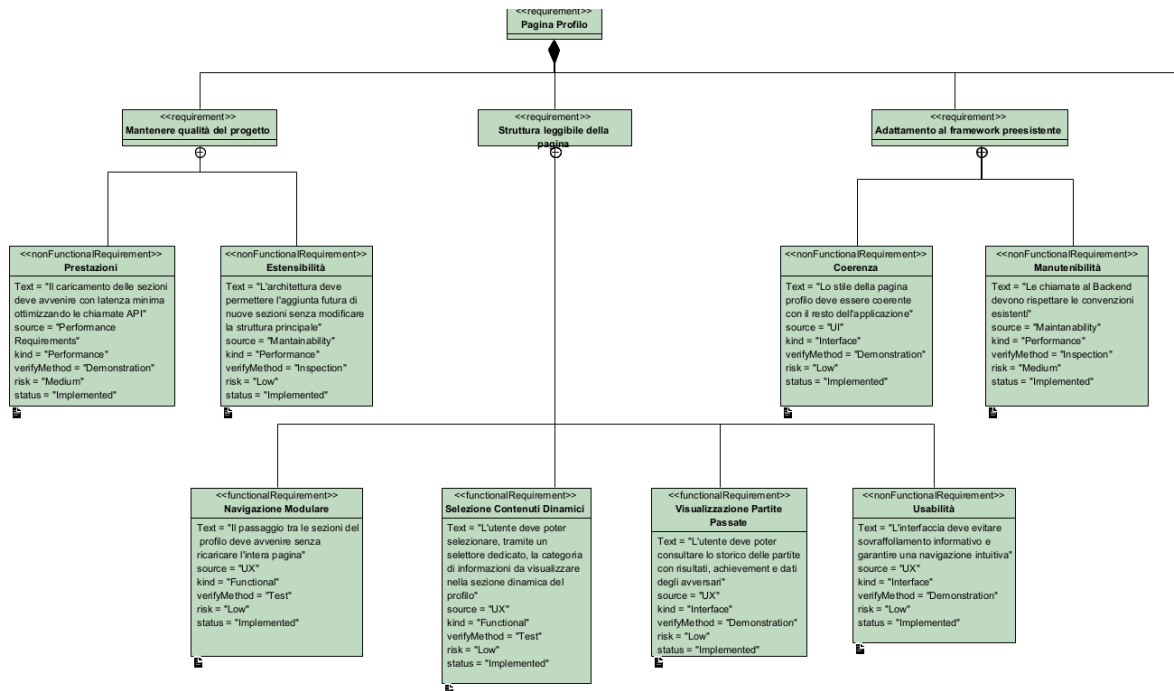


Figura 1 Diagramma dei componenti del sistema e interazioni tra i moduli 1

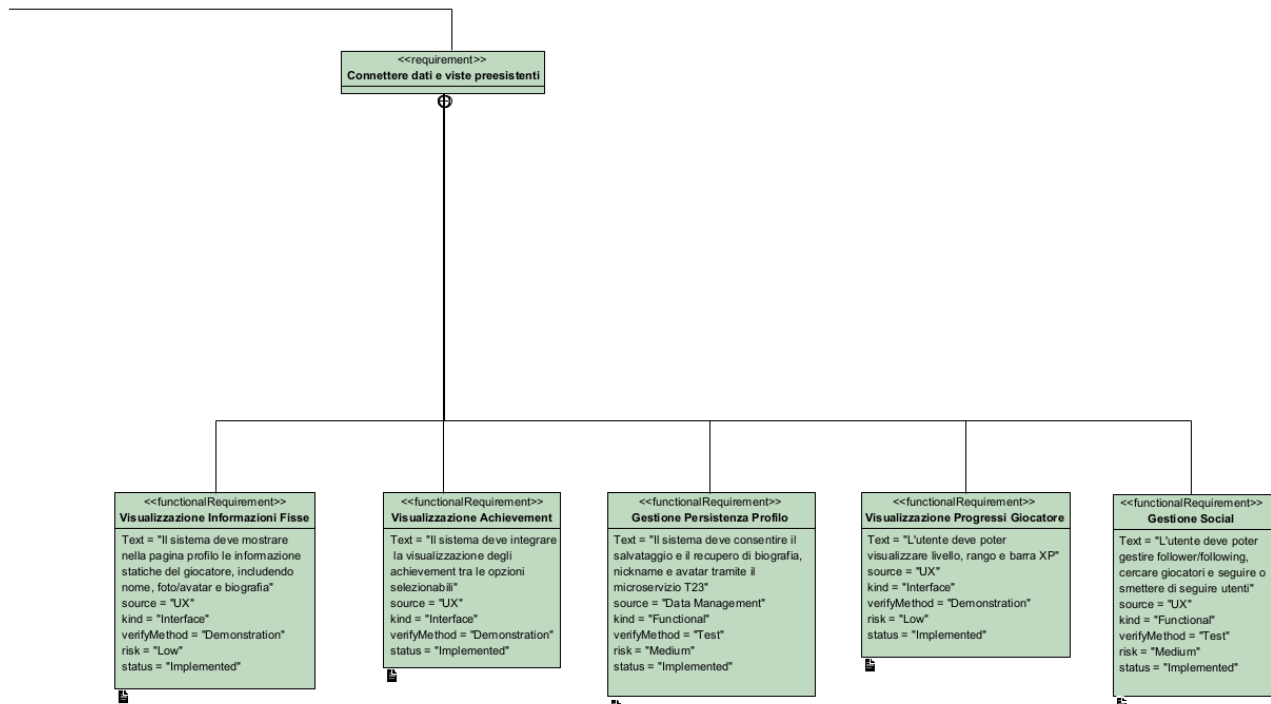


Figura 2 Diagramma dei componenti del sistema e interazioni tra i moduli 2

Si sono individuate, in particolare, quattro macroaree da modificare ed implementare:

Qualità del progetto:

Questo pilastro impone vincoli rigorosi di prestazioni ed estensibilità.

L'obiettivo è garantire che l'integrazione asincrona delle API non generi colli di bottiglia, predisponendo il sistema a futuri ampliamenti modulari senza degradarne le performance.

Esperienza d'uso:

Questa area formalizza la transizione verso una navigazione modulare e la selezione di contenuti dinamici. L'adozione del paradigma *Single Page Experience* è fondamentale per superare la staticità delle versioni precedenti, migliorando drasticamente la reattività del sistema e il comfort di navigazione.

Adattamento al Framework:

L'efficacia del modello risiede nella sua capacità di integrarsi senza frizioni nel microservizio T23, questo requisito assicura non solo la coerenza stilistica con il resto della piattaforma, ma anche la manutenibilità a lungo termine del codice di back-end.

Connettività dati:

Questa sezione costituisce il cuore funzionale del task, poiché essa gestisce il recupero e la visualizzazione dello storico partite, dei progressi e delle dinamiche social (*Follower/Following*), definendo per ogni elemento precisi metodi di verifica e analisi del rischio, garantendo così una tracciabilità totale del progetto.

1.1.1 Obiettivi principali del progetto:

Mantenere l'elevata qualità del codice:

Le nuove modifiche devono rispettare gli standard di code quality già presenti nel progetto, evitando regressioni o soluzioni incoerenti.

Migliorare l'esperienza utente

La pagina profilo deve risultare:

- Leggibile.
- Intuitiva.
- Facile da navigare.

Avere una coerenza architetturale, per cui le eventuali modifiche in T5 (e parzialmente in T23) devono essere coerenti con il resto del progetto, seguendo i pattern architetturali e stilistici già consolidati.

Unificazione delle funzionalità del profilo

Attualmente le sezioni Edit, Teams e Achievements sono separate, per cui l'obiettivo è di unirle in un'unica pagina coesa e rendere visibili tutti i dati logicamente connessi al profilo utente nello stesso contesto.

1.1.2 Elenco dei requisiti

Di seguito è riportato l'elenco completo dei requisiti individuati:

Requisiti funzionali (RF)

Codice	Nome	Descrizione
RF1	Visualizzazione Informazioni Fisse	Il sistema deve mostrare nella pagina profilo le informazioni statiche del giocatore, includendo nome, foto/avatar e biografia.
RF2	Selezione Contenuti Dinamici	L'utente deve poter selezionare, tramite un selettore dedicato, la categoria di informazioni da visualizzare nella sezione dinamica del profilo.
RF3	Visualizzazione Achievement	Il sistema deve integrare la visualizzazione degli achievement tra le opzioni selezionabili.
RF4	Gestione Persistenza Profilo	Il sistema deve consentire il salvataggio e il recupero di biografia, nickname e avatar tramite il microservizio T23.
RF5	Navigazione Modulare	Il passaggio tra le sezioni del profilo deve avvenire senza ricaricare l'intera pagina.
RF6	Visualizzazione Progressi Giocatore	L'utente deve poter visualizzare livello, rango e barra XP.
RF7	Visualizzazione Partite Passate	L'utente deve poter consultare lo storico delle partite con risultati, achievement e dati degli avversari.
RF8	Gestione Social	L'utente deve poter gestire follower/following, cercare giocatori e seguire o smettere di seguire utenti.

Tabella 1 Requisiti funzionali

Requisiti non funzionali (RNF)

Codice	Nome	Descrizione
RNF1	Estensibilità	L'architettura deve permettere l'aggiunta futura di nuove sezioni senza modificare la struttura principale.
RNF2	Usabilità	L'interfaccia deve evitare sovraffollamento informativo e garantire una navigazione intuitiva.
RNF3	Coerenza	Lo stile della pagina profilo deve essere coerente con la pagina Achievement.
RNF4	Manutenibilità	I modelli dati del microservizio T23 devono rispettare le convenzioni esistenti.
RNF5	Prestazioni	Il caricamento delle sezioni deve avvenire con latenza minima ottimizzando le chiamate API.

Tabella 2 Requisiti non funzionali

1.2 Progettazione della soluzione

Come prima cosa bisogna comprendere l'architettura a microservizi di *TestRobotChallenge*.

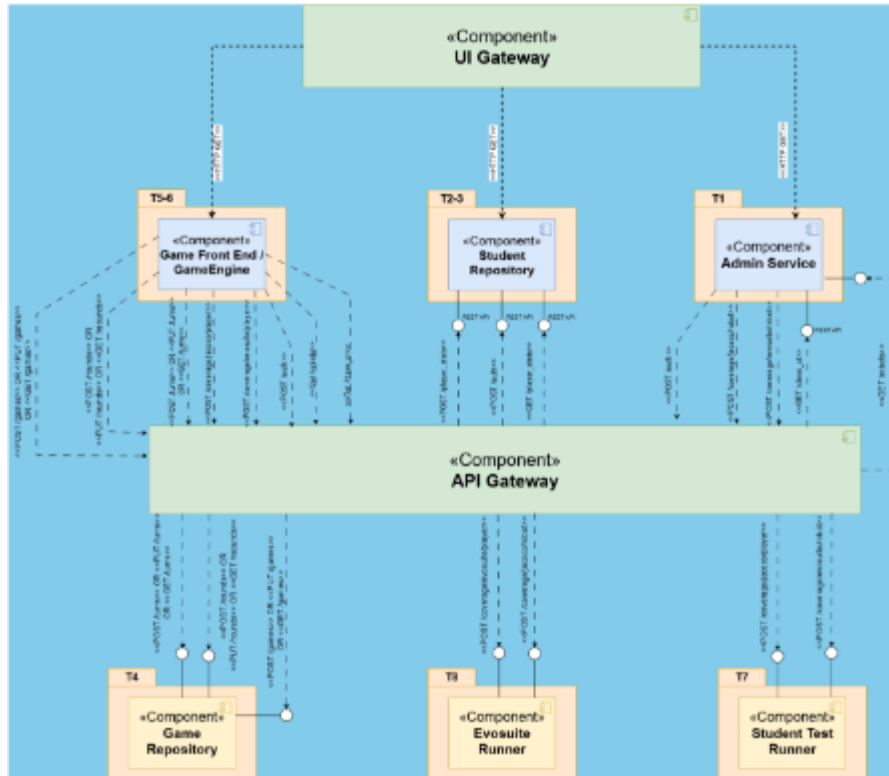


Figura 3 Component Diagram

In un'architettura a microservizi ogni componente è separato, indipendente ed offre servizi specializzati (uno o al massimo due). La Task R6 si focalizzerà principalmente su T5 (che contenente il Front-End per il giocatore) e T23 (contenente i dati del giocatore).

1.2.1 Analisi dell’impatto

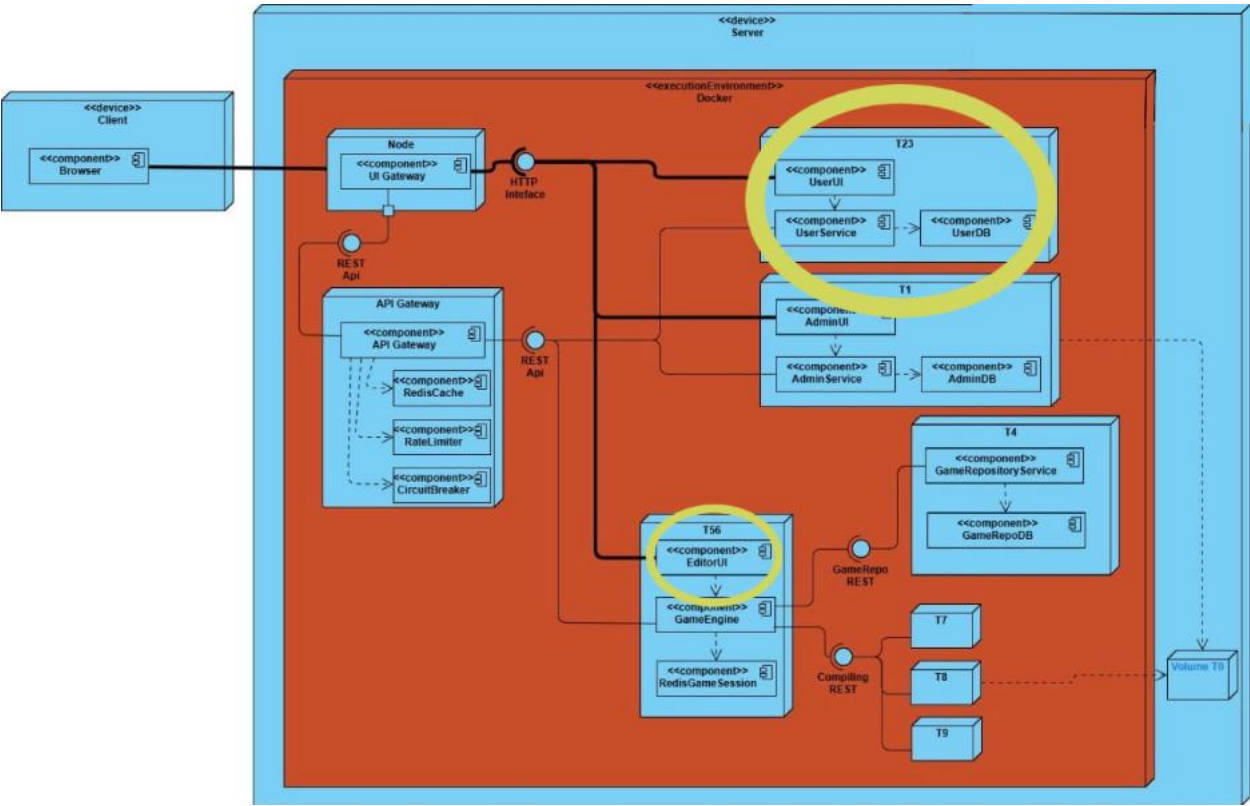


Figura 4 Blocchi impattanti

Qui si definiscono le modifiche impattanti sul progetto e per una miglior comprensibilità s'introduce qui di seguito una tabella dove si analizza nel dettaglio le azioni impattanti sui vari livelli.

Livello	Componente	Natura dell'impatto
Presentazione	UI del profilo	ALTO: rifacimento completo dell'interfaccia utente e della logica CSS.
Coordinamento	Controller T5	MEDIO: introduzione di nuovi endpoint per la gestione asincrona dei dati.
Logica	Servizi T23	MEDIO/ALTO: aggiunta di nuovi servizi per storico partite e funzionalità social.
Integrazione	Servizi T23/T5	BASSO: aggiornamento delle chiamate REST interne tra microservizi.

Tabella 3 Cambiamenti

1.2.2 Pattern da seguire

L'architettura preesistente utilizza *Spring Framework* e quindi segue il pattern architetturale *MVC (Model – View – Controller)*.

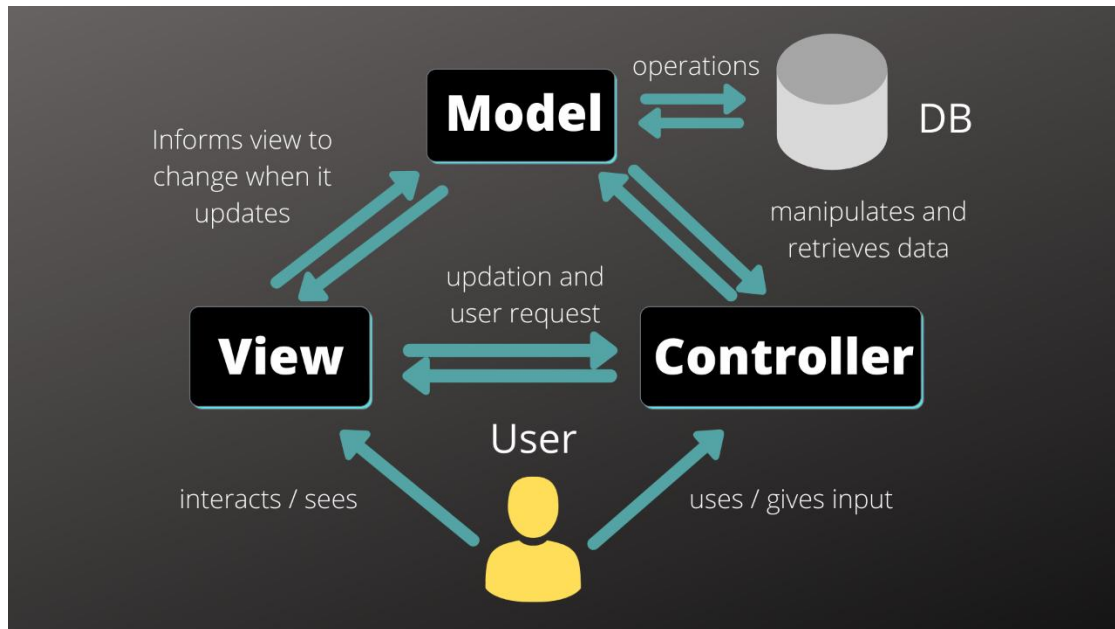


Figura 5 Pattern architetturale

Per rimanere coerenti con il resto del contesto (e quindi soddisfare la macroarea *adattamento al framework*) la soluzione dovrà seguire questo pattern.

In particolare, quindi, si prevede:

- In T5 un file HTML con supporto JS per la parte View del pattern, dedicato esclusivamente alla presentazione dei dati. (già presenti nell'architettura AS-IS ma che hanno bisogno di pesanti modifiche dato che non seguono il pattern).
- Sia in T5 che in T23 verranno utilizzati dei Controller già esistenti, ma con probabili aggiunte di rotte per una più completa gestione dei dati profilo.
- In T23 si dovrà controllare la cartella Model, che nel *framework Spring* contiene le varie classi java che identificano le entità (nel caso da migliorare).

1.3 Studio del database in T23

I dati del profilo giocatore sono gestiti nel container T23, che utilizza un database MySQL già consolidato. Per osservare le varie tabelle e il diagramma complessivo, è stato utilizzato il plugin database dell'IDE IntelliJ.

1.3.1 Configurazione iniziale del container

Prima di poter accedere al database in locale, è stato necessario modificare il file docker-compose.yml, il manifest che descrive il deploy di Docker in runtime, in particolare quello del microservizio T23:

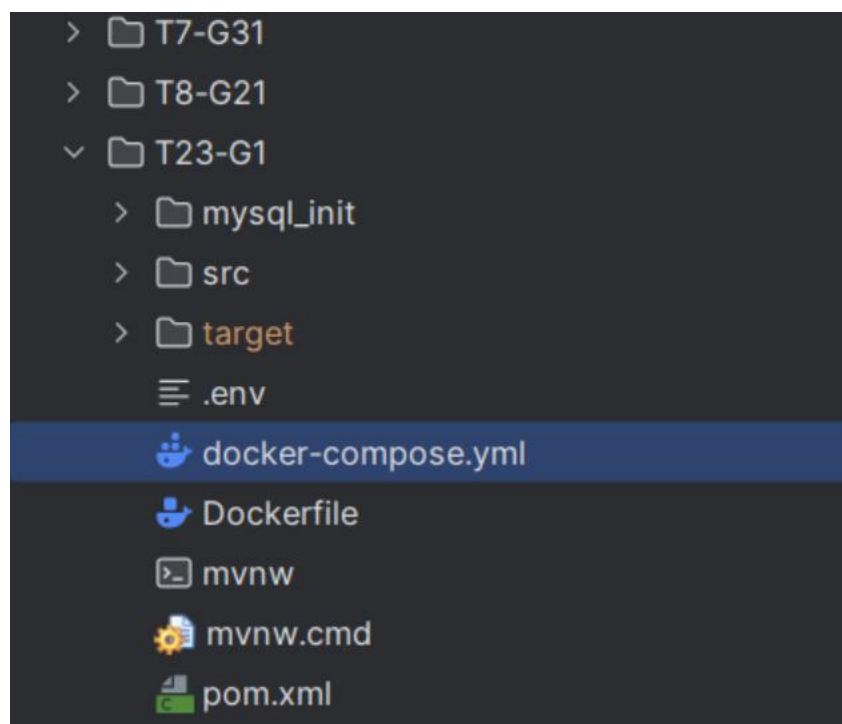


Figura 4 Configurazione del container

1.3.2 Accesso al database tramite IntelliJ

Modificando il file docker-compose.yml in questo modo (prima era expose: 3306), è possibile osservare il database direttamente tramite IntelliJ:

```
1  name: t23
2
3  volumes:
4    mysql_data_volume:
5
6  services:
7    db:
8      image: mysql:latest
9      container_name: t23-mysql_db
10     environment:
11       MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
12       MYSQL_DATABASE: ${MYSQL_DATABASE}
13       MYSQL_USER: ${MYSQL_USER}
14       MYSQL_PASSWORD: ${MYSQL_PASSWORD}
15     volumes:
16       - ./mysql_init/init.sql:/docker-entrypoint-initdb.d/init.sql
17       - mysql_data_volume:/var/lib/mysql
18     ports:
19       - "3306:3306"
20     healthcheck:
21       test: [ "CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-u", "${MYSQL_USER}", "-p${MYSQL_PASSWORD}" ]
22       interval: 30s
23       timeout: 60s
24       retries: 5
25       start_period: 80s
26     networks:
27       - global-network
28     restart: always
```

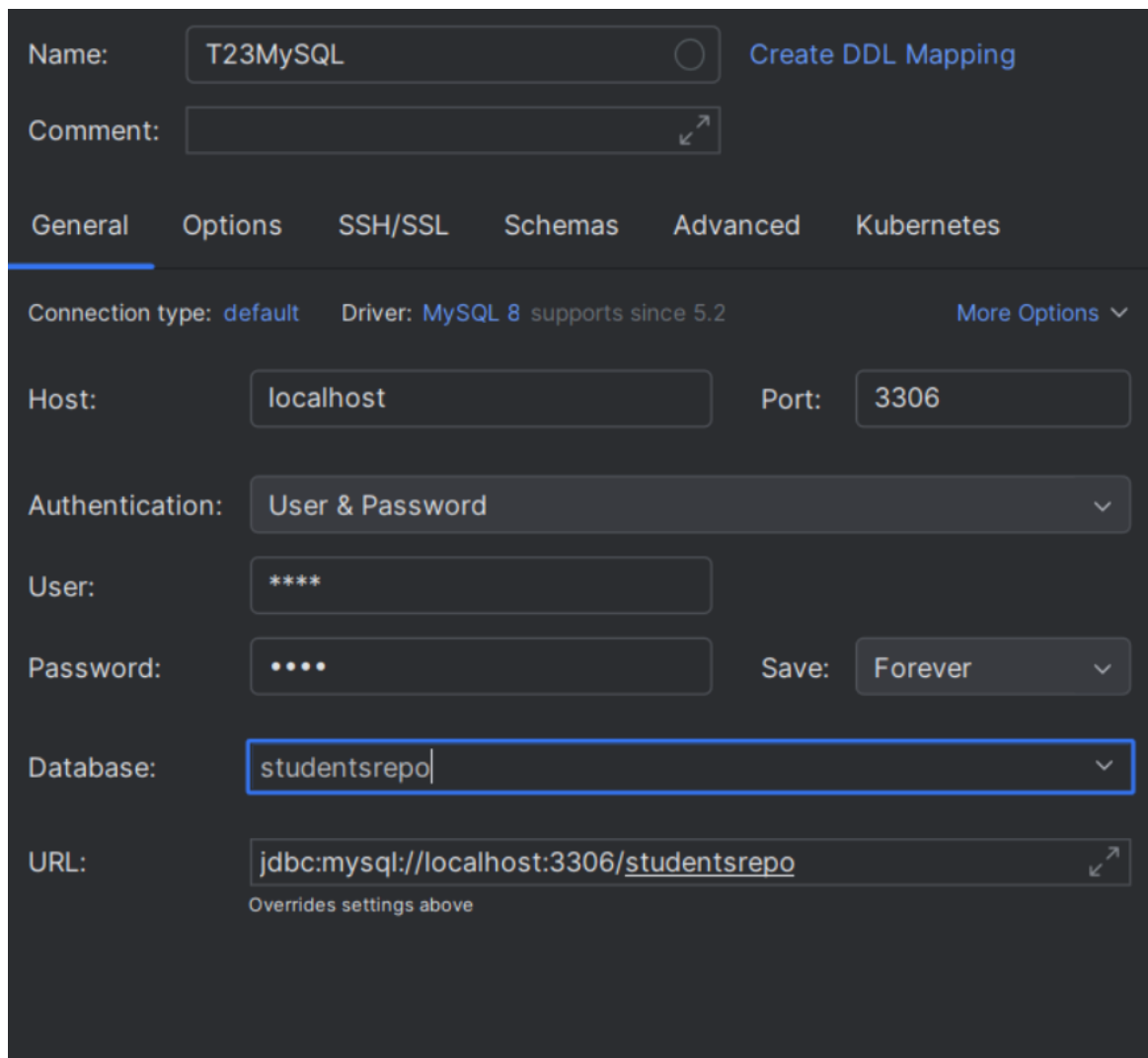
Figura 5 Accesso al database

1.3.3 Connessione tramite plugin database

Qui si possono analizzare le modifiche apportate, ovvero nel menu del plugin, aggiungere un nuovo database e specificare:

- La **porta** indicata nel file .yaml.
- **Credenziali valide** per il database.
- Il **nome del database** definito nel file .env.

A questo punto, è possibile navigare e osservare i dati.



The image shows a dark-themed configuration window for a database connection. At the top, there's a 'Name' field with 'T23MySQL' and a 'Create DDL Mapping' button. Below it is a 'Comment' field. A tabbed interface follows with 'General', 'Options', 'SSH/SSL', 'Schemas', 'Advanced', and 'Kubernetes'. The 'General' tab is active, showing 'Connection type: default' and 'Driver: MySQL 8 supports since 5.2'. The 'Host' is 'localhost' and 'Port' is '3306'. 'Authentication' is set to 'User & Password'. The 'User' field contains '****' and the 'Password' field contains '....'. A 'Save' dropdown is set to 'Forever'. The 'Database' dropdown is set to 'studentsrepo'. The 'URL' field shows 'jdbc:mysql://localhost:3306/studentsrepo' with a note 'Overrides settings above'.

Figura 6 Navigazione del database

1.3.4 Visualizzazione diagrammi del database

Il plugin offre la funzionalità molto utili per la generazione di diagrammi e delle tabelle automaticamente:

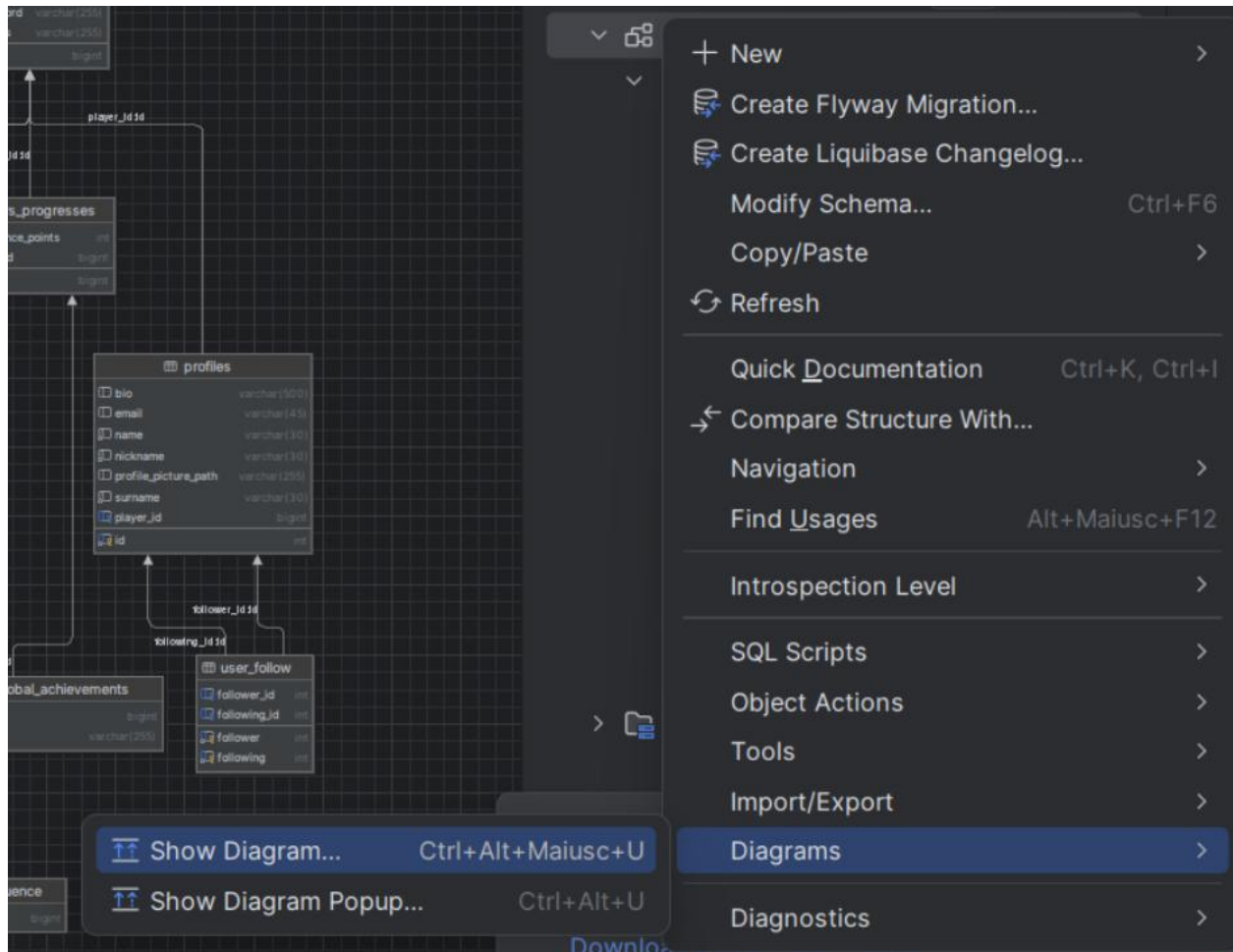


Figura 7 Diagramma generato automaticamente dal plugin

Il diagramma risultante mostra le varie tabelle e le relazioni tra di esse. Pur non mostrando tutti i dettagli, si può osservare la struttura preesistente, in particolare le informazioni relative al giocatore. Ciò è stato utile per iniziare lo sviluppo della pagina profilo.

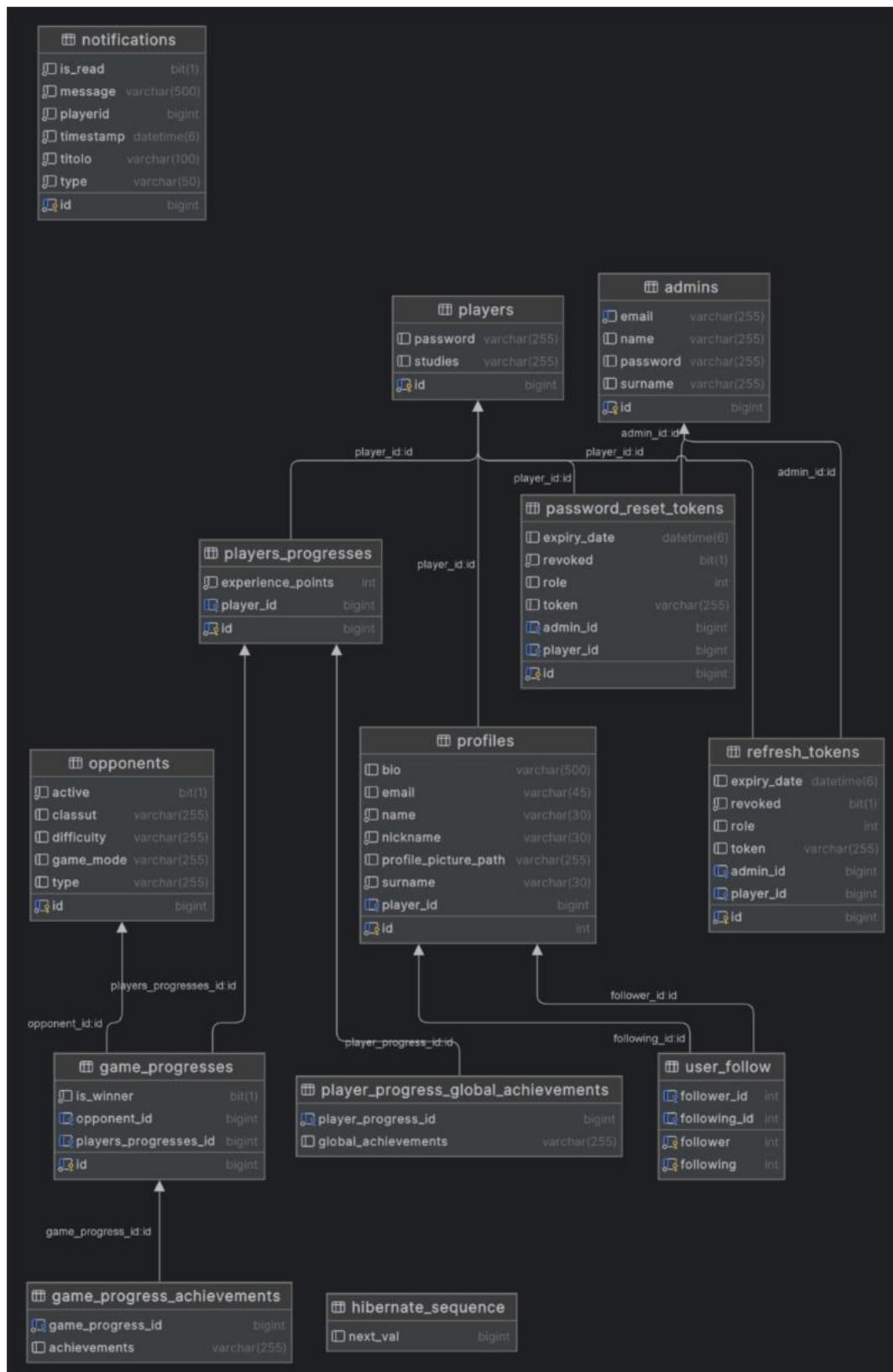


Figura 8 Schema del database - Gestione profilo giocatore

1.4 Entità principali

players

La tabella players rappresenta l'utente registrato sulla piattaforma ed è il fulcro dell'intero schema relazionale.

- id (*PK*): identificativo univoco del giocatore.
- password: password cifrata.
- studies: informazioni relative al percorso di studi del giocatore.

Tale entità viene utilizzata come riferimento principale per i progressi del profilo e per i meccanismi di autenticazione.

profiles

La tabella profiles gestisce i dati estesi del profilo utente, separandoli dai dati di autenticazione.

- id (*PK*).
- player_id (*FK* → *players.id*).
- name, surname.
- nickname.
- email.
- bio.
- profile_picture_path.

1.4.1 Gestione dei progressi del giocatore

players_progresses

La tabella players_progresses rappresenta lo stato di avanzamento complessivo del giocatore all'interno della piattaforma.

- id (*PK*).
- player_id (*FK* → *players.id*).
- experience_points: punti esperienza accumulati.

Essa funge da collegamento tra il giocatore e le singole partite giocate.

game_progresses

La tabella game_progresses descrive una singola partita o sfida sostenuta dal giocatore.

- id (*PK*).
- players_progresses_id (*FK* → *players_progresses.id*).
- opponent_id (*FK* → *opponents.id*).
- is_winner: indica l'esito della partita.

opponents

La tabella opponents definisce gli avversari automatizzati (Robot) contro cui il giocatore può competere.

- id (*PK*).
- type.
- difficulty.
- class.
- game_mode.
- active.

1.4.2 Sistema di achievement

game_progress_achievements

Contiene gli achievement ottenuti durante una specifica partita.

- game_progress_id (*FK* → *game_progresses.id*).
- achievements.

player_progress_global_achievements

Memorizza gli achievement globali associati al progresso complessivo del giocatore.

- player_progress_id (*FK* → *players_progresses.id*).
- global_achievements.

La separazione tra achievement locali e globali consente una gestione più flessibile degli obiettivi di gioco.

1.4.3 Funzionalità social

user_follow

La tabella user_follow implementa il meccanismo di follow tra utenti della piattaforma.

- follower_id (*FK* → *profiles.id*).
- following_id (*FK* → *profiles.id*).
- follower.
- following.

Questa struttura abilita funzionalità social come la visualizzazione dei profili seguiti e dei follower.

1.4.4 Notifiche

notifications

La tabella notifications gestisce le notifiche inviate ai giocatori.

- id (*PK*).
- player_id (*FK* → *players.id*).
- title.
- message.
- type.
- is_read.
- timestamp.

(non completamente implementata)

1.4.5 Sicurezza e autenticazione

admins

Rappresenta gli amministratori del sistema.

- id (*PK*).
- name, surname.
- email.
- password.

password_reset_tokens

Gestisce il processo di reset della password per giocatori e amministratori.

- id (*PK*).
- token.
- expiry_date.
- revoked.
- role.
- player_id (*FK*).
- admin_id (*FK*).

refresh_tokens

Supporta il meccanismo di autenticazione tramite token JWT.

- id (*PK*).
- token.
- expiry_date.
- revoked.
- role.
- player_id (*FK*).
- admin_id (*FK*)

1.5 Class diagram

Il diagramma fornito automaticamente da IntelliJ mostra lo schema del database, di seguito è riportato il diagramma delle classi riguardanti il profilo utente presenti in T23.

Si è deciso di mantenere questa struttura, essendo adatta ai requisiti posti dalla task.

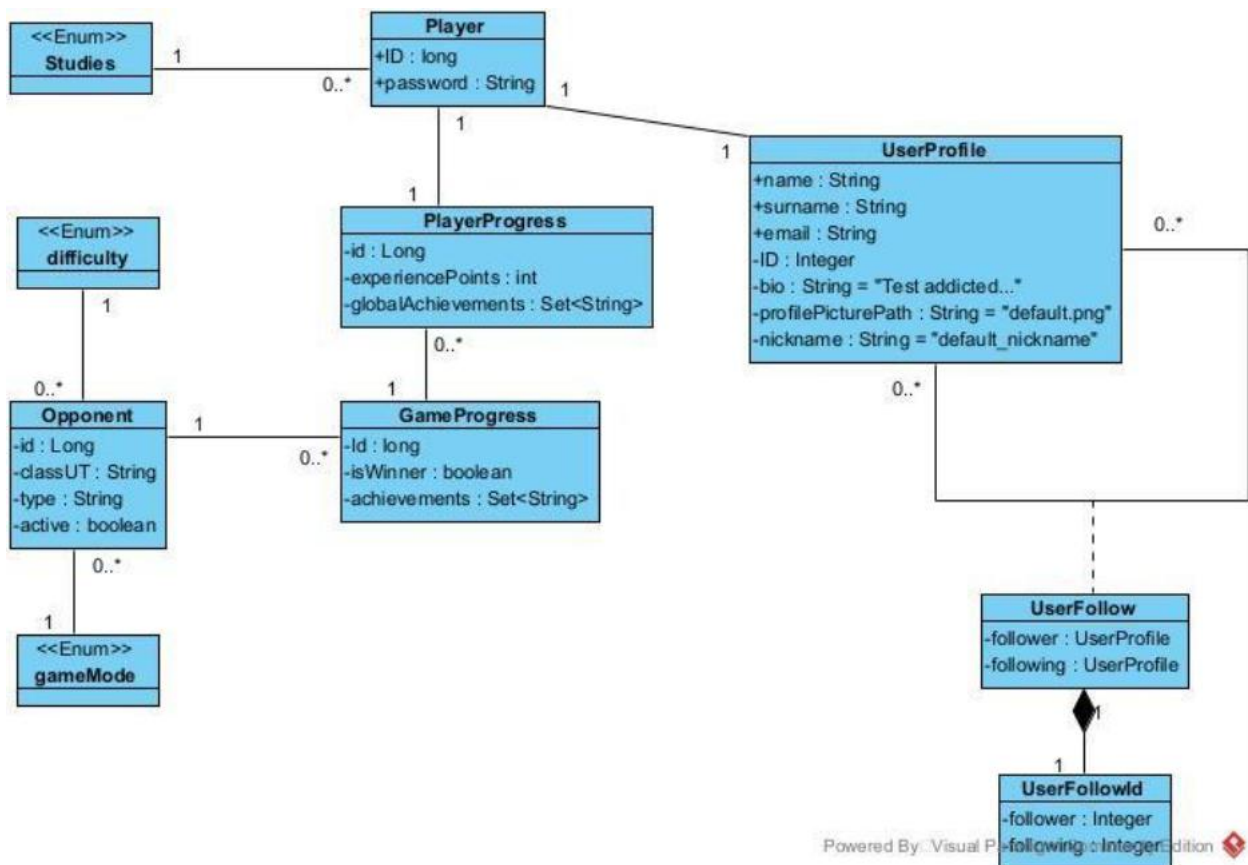


Figura 9 Diagramma delle classi

Il diagramma delle classi illustra la struttura logica del microservizio T23, focalizzandosi su come il sistema modella l'identità del giocatore e la sua evoluzione nel gioco. Al centro troviamo la separazione tra l'entità *Player* (classe), dedicata esclusivamente all'autenticazione, e la classe *UserProfile*, che ne rappresenta l'identità pubblica (nickname, bio, avatar).

Questa distinzione 1:1 è fondamentale per la privacy e per rendere il profilo facilmente espandibile in futuro senza intaccare le credenziali di accesso.

La sezione dedicata alla gamification collega il giocatore ai suoi successi tramite *PlayerProgress*, che centralizza i punti esperienza e i traguardi globali.

Lo storico delle singole partite è invece gestito da *GameProgress*, che traccia l'esito di ogni sfida contro i vari opponent (i Robot), specificandone difficoltà e modalità.

Infine, il diagramma modella il grafo sociale attraverso una relazione riflessiva su *UserProfile*, mediata dalla classe *UserFollow*. Quest'ultima, grazie a una chiave composta assicura l'integrità dei legami di "follow" tra gli utenti, garantendo che ogni interazione sociale sia tracciata in modo univoco e coerente.

IntelliJ aveva considerato attributi con la classe *Set* come vere e proprie classi separate, inoltre aveva ommesso i vari Enum utilizzati. Questo diagramma è più chiaro, mostra ciò che è presente nel database e come sono organizzati le tavole (gli achievements, per esempio, sono semplici Set di stringhe e non entità separate) ed è più chiara la struttura di *UserFollow*.

Questa entità è relazionale, per cui ha come attributi i due *UserProfile* a cui è collegata e ha come chiave unica la coppia di *Integer* (follower, following) ovvero gli ID dei due profili.

1.5.1 Il Nucleo dell'identità Player e UserProfile

Al centro del diagramma si colloca la classe *Player*, che funge da entità radice per l'autenticazione e l'identificazione dell'utente, essa contiene attributi critici quali l'identificativo univoco (ID) e la password.

Relazione con UserProfile:

È presente una relazione di associazione diretta tra *Player* ed *UserProfile*, questa separazione rappresenta una precisa scelta di design architetturale, volta a isolare:

- I dati di accesso e autenticazione (*Player*)
- Le informazioni anagrafiche e pubbliche (*UserProfile*), come nickname, bio e avatar.

Tale approccio facilita la gestione della privacy e rende il profilo facilmente estendibile in futuro.

Contesto accademico:

La classe *Player* è inoltre associata all'enumerazione *Studies* (relazione **1:0..***).

1.5.2 Gamification e Persistenza: PlayerProgress e GameProgress

Questo dominio gestisce la logica del punteggio e dei traguardi raggiunti.

PlayerProgress:

Collegata al Player tramite una relazione diretta, questa classe centralizza lo stato globale del giocatore, essa include:

- L'accumulo totale dei punti esperienza (*experiencePoints*).
- Il set di achievement globali sbloccati (*globalAchievements*).

GameProgress:

Rappresenta lo storico delle singole sessioni di gioco, ogni istanza di *PlayerProgress* può non essere associata a nessuna istanza o a più istanze di *GameProgress*, ciascuna però delle quali registra:

- L'esito della partita (*isWinner*).
- Gli achievement specifici ottenuti durante quella sessione di gioco.

Relazione con opponent:

Ogni partita è associata a un opponent (avversario), caratterizzato da parametri tecnici quali:

- La classe sotto test (*classUT*).
- Il tipo di robot.
- Il livello di difficoltà, modellato tramite una specifica enumerazione, che consente di rappresentare in modo strutturato le diverse tipologie di sfida offerte dalla piattaforma.

1.5.3 Organizzazione Social

Il grafo sociale: *UserFollow* e *UserFollowId*

Questa sezione modella le relazioni di follow tra utenti.

- Ricorsività del profilo.
- La classe *UserProfile* presenta un'auto-associazione (relazione riflessiva **0..*** ↔ **0..***), che indica come un profilo possa seguire ed essere seguito da altri profili.

Modellazione della Relazione

La relazione di follow è mediata dalla classe *UserFollow*, che funge da tabella di associazione, essa utilizza una chiave composta definita in *UserFollowId*, contenente:

- L'identificativo del follower (chi segue).
- L'identificativo del following (chi è seguito).

Composizione

L'uso della relazione di composizione tra *UserFollow* e *UserFollowId* indica che l'identità della relazione di follow non può esistere senza i riferimenti ai due utenti coinvolti, ciò garantisce l'integrità referenziale del grafo sociale.

1.6 Analisi pagina AS-IS

Prima dell'intervento di implementazione, la pagina profilo del framework *TestingRobotChallenge* si presentava come una struttura informativa essenziale, ma profondamente frammentata sia nell'esperienza d'utilizzo utente sia nell'architettura dei dati. L'interfaccia originale, servita dal *UserProfileController* del microservizio T5 (tramite endpoint */profile*), era concepita per offrire una visione statica di pochi parametri, risultando inadeguata rispetto ai moderni standard di gamification.

Le seguenti figure illustrano la configurazione originale, evidenziando i limiti che hanno motivato il successivo rifacimento modulare:

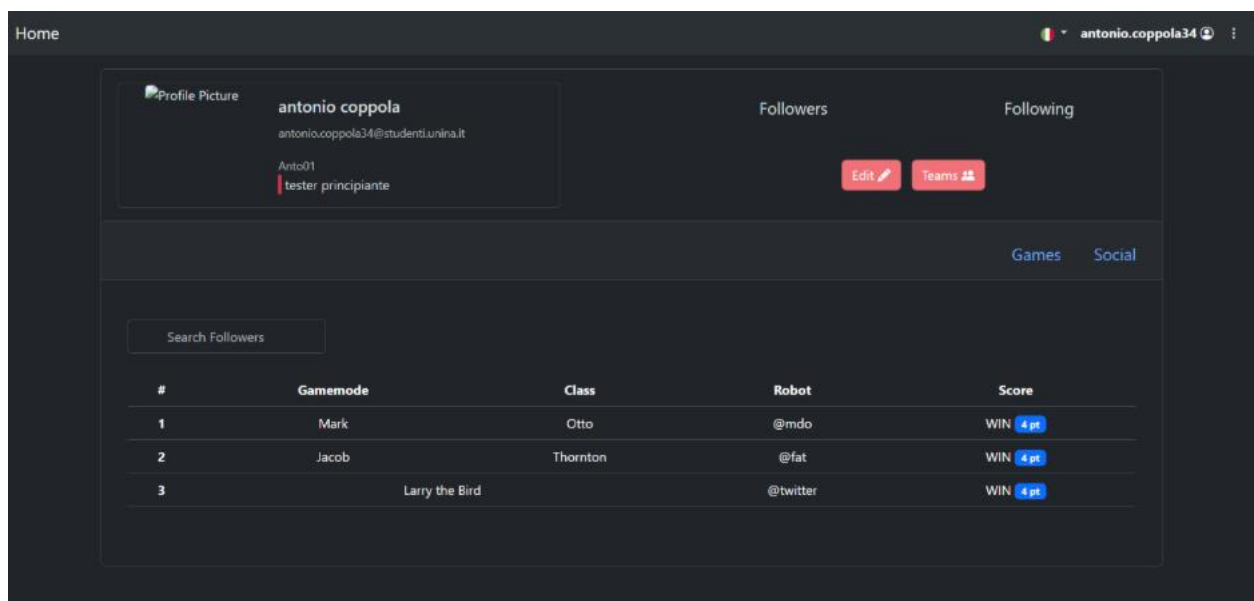


Figura 10 Pagina profilo iniziale 1

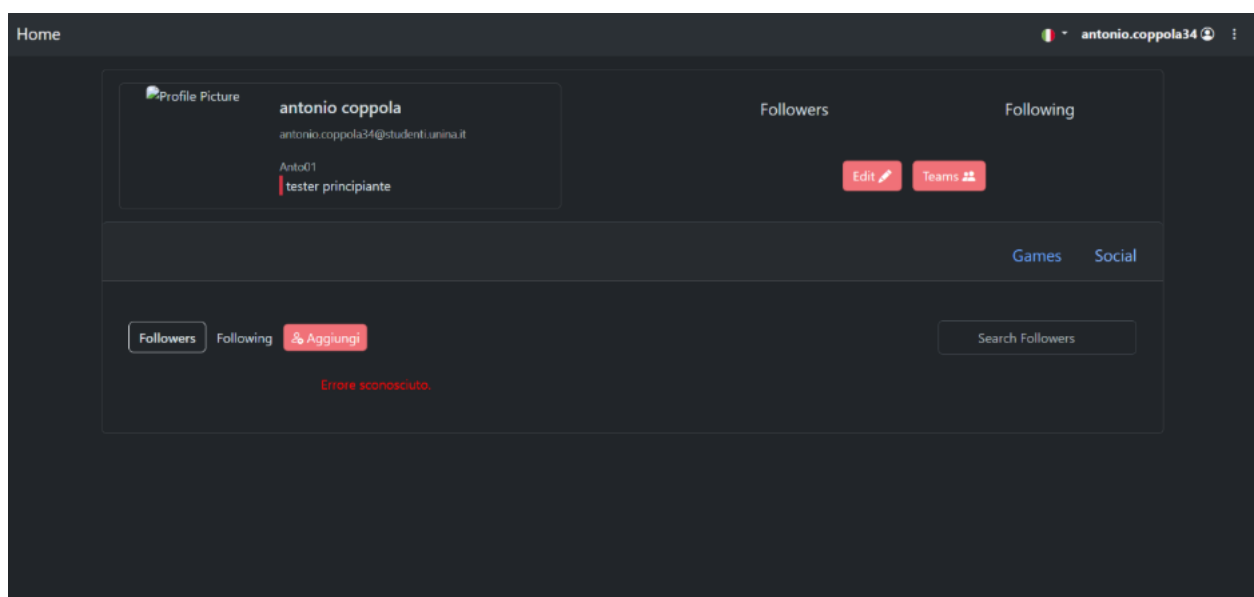


Figura 11 Pagina profilo iniziale 2

1.6.1 Analisi delle criticità dello stato attuale

La struttura preesistente risultava poco coesa e incapace di supportare un ecosistema competitivo date le seguenti argomentazioni.

Progressi separati dal resto del profilo

Il sistema gestiva i punti esperienza (XP) insieme agli achievements, i quali venivano visualizzati in un'altra pagina, accessibile dalla pagina principale del sito. Quindi il giocatore doveva continuamente cambiare pagina per visualizzare i suoi dati personali.

Navigazione sociale instabile

Le sezioni *Follower* e *Following* presentavano frequenti errori di caricamento (es. "Errore sconosciuto"), sintomo di una gestione asincrona non ottimizzata le due. Inoltre, la presenza di pulsanti decontestualizzati come "Edit" e "Teams" rendeva la navigazione confusa.

Isolamento logico

Una delle violazioni più gravi dell'usabilità riguardava l'accessibilità, ovvero la pagina profilo non era raggiungibile tramite i menu di navigazione dell'applicazione, ma richiedeva l'inserimento manuale dell'URL nel browser, rendendo poco intuitivo l'utilizzo del sistema all'utente.

Criticità tecniche e architetturali

Sotto il profilo ingegneristico, il sistema presentava un elevato debito tecnico e una scarsa applicazione del principio di "*separation of concerns*", per cui:

- Mancata persistenza e dati Hardcoded:

Il sistema non garantiva una reale integrazione con il database per la visualizzazione dinamica, per cui, molte informazioni visualizzate erano semplici *placeholder* grafici (valori statici inseriti direttamente nell'HTML/JS), privi di valore informativo reale e scollegati dalle reali sessioni di gioco dell'utente.

- Incoerenza nei contratti API:

È emersa una grave lacuna nella comunicazione tra i microservizi T5 e T23, i vari tipi di ritorno e/o endpoint tra i due microservizi nelle loro comunicazioni non combaciavano.

```

237 MultiValueMap<String, String> formData = new LinkedMultiValueMap<>();
238 formData.add("jet", "jet");
239 // Chiamata POST utilizzando il metodo della classe base
240 return callRestPost(endpoint, formData, null, JetValidationResponseDTO.class);
241 }
242
243 // Metodo per ottenere la lista degli utenti
244 private List<User> getUsers() {
245     final String endpoint = "/players/searchStudents";
246     return callRestGET(endpoint, null, new ParameterizedTypeReference<List<User>>() {
247     });
248 }
249
250 private User getUser(String userId) {
251     final String endpoint = "/players/students_list/" + userId;
252     return callRestGET(endpoint, null, User.class);
253 }
254
255 //Do una lista di ID e mi ritorna una lista di User
256 // Implementata a mano perchè un po' strana è una POST che ottiene dati come una GET
257 private List<User> getUserByList(List<String> idsStudenti) {
258     final String endpoint = "/student/getStudentiTeam";
259     // Crea un oggetto HttpEntity con i dati che vogliamo inviare (la lista degli ID)
260     HttpEntity<List<String>> requestEntity = new HttpEntity<> (idsStudenti);
261     // Esegui la chiamata POST all'endpoint
262     ResponseEntity<> responseEntity = restTemplate.exchange(
263         BASE_URL + "/" + SERVICE_PREFIX + endpoint, // URL dell'endpoint
264         HttpMethod.POST, // Tipo di richiesta POST
265         requestEntity, // Corpo della richiesta (lista di studenti)
266         new ParameterizedTypeReference<List<User>>() {
267         } // Tipo di risposta che ci aspettiamo
268     );
269 }

```

Figura 12 Endpoint usato in GetUser (T5)

```

46 public class PlayerController {
47     @GetMapping("/players/studentByEmail/{emailStudente}") no usages
48     @ResponseBody
49     public ResponseEntity<> getStudentByEmail(@PathVariable("emailStudente") String emailStudente) {
50         return playerService.getStudentByEmail(emailStudente);
51     }
52
53     //Modifica 12/12/2024
54     @GetMapping("/players/studentsByNameSurname") no usages
55     @ResponseBody
56     public List<Map<String, Object>> getStudentsBySurnameAndName(@RequestBody Map<String, String> request) {
57         return playerService.getStudentsBySurnameAndName(request);
58     }
59
60     //Modifica 12/12/2024 Giuseppe: Aggiunta nuova rotta che verrà aggiunta per la ricerca degli studenti.
61     @PostMapping("/players/searchStudents") no usages
62     @ResponseBody
63     public List<Map<String, Object>> searchStudents(@RequestBody Map<String, String> request) {
64         return playerService.searchStudents(request);
65     }
66
67     @GetMapping("/students/students_list/{id}") no usages
68     @ResponseBody
69     public Player getStudent(@PathVariable String ID) { return playerService.getUserByID(Long.parseLong(ID)); }
70 }

```

Figura 13 Endpoint effettivo in T23

```
// METODO PER ESTRARRE USER ID DAL JWT
private static Long extractUserId(String jwt) {
    try {
        if (jwt == null || jwt.split("\\.").length < 2) {
            throw new IllegalArgumentException("JWT non valido: formato errato");
        }
        byte[] decodedBytes = Base64.getDecoder().decode(jwt.split("\\.")[1]);
        String decodedJson = new String(decodedBytes, StandardCharsets.UTF_8);
        Map<String, Object> payload = OBJECT_MAPPER.readValue(decodedJson, Map.class);
        Object userId = payload.get("userId");
        if (userId == null) {
            throw new IllegalArgumentException("JWT non valido: userId mancante");
        }
        return Long.parseLong(userId.toString());
    } catch (Exception e) {
        logger.error("[PageBuilder] Errore nella decodifica del JWT: {}", e.getMessage());
        return null;
    }
}
```

Figura 14 Funzione per l'estrazione dell'ID utente del token, restituisce un Long

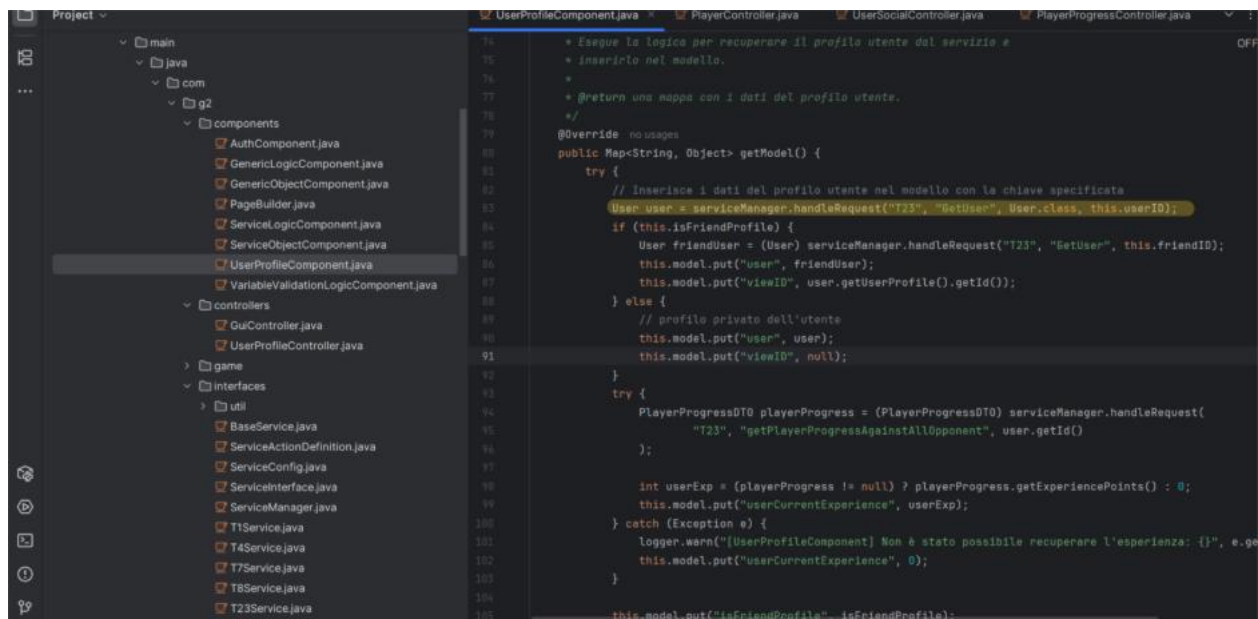


Figura 15 UserId viene passato a GetUser senza essere convertito da Long a String

- Mancata separazione tra html e javascript:

Numerose funzionalità risultavano inserite nel template HTML mentre il file JS dedicato era scarso, la mancata separazione tra grafica e logica rendeva complessa la manutenzione. La gestione incoerente degli stati della UI causava inoltre sovrapposizioni visive errate, come la comparsa di strumenti di ricerca sociale all'interno della vista dello storico partite.

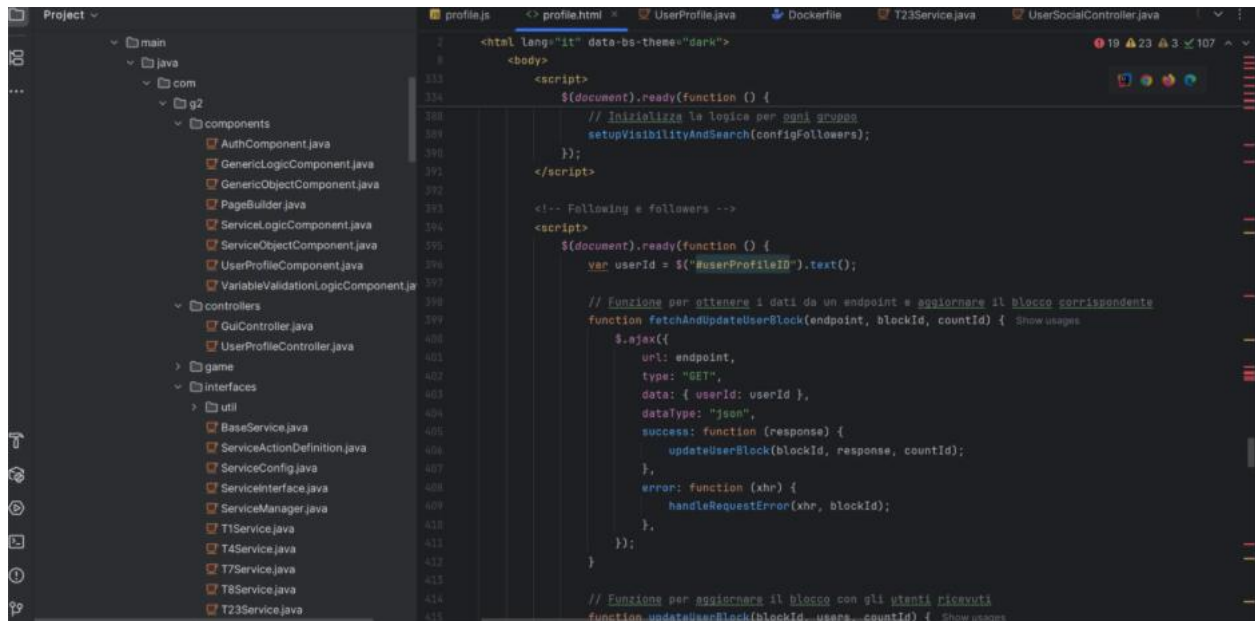


Figura 16 Analisi del grado di accoppiamento tra la logica di presentazione (HTML) e la logica applicativa (JS)

In sintesi, il lavoro svolto con il task R6 ha avuto come obiettivo la trasformazione di questo semplice visualizzatore statico in una *single page experience* dinamica e robusta, risolvendo i conflitti di tipo e garantendo la reale persistenza dei dati.

Tabella dei problemi riscontrati

L'analisi delle criticità emerse è sintetizzata nella Tabella 4, che riassume in modo strutturato i principali problemi architetturali e di design riscontrati nella versione “*legacy*” della pagina profilo.

Per ciascuna area di intervento vengono evidenziati l'ambito coinvolto, la natura del problema e il relativo impatto sull'architettura complessiva del sistema. Questa panoramica consente di comprendere immediatamente le motivazioni alla base delle decisioni progettuali adottate nel task R6.

Categoria	Campo	Descrizione del problema	Impatto sull'architettura
Accessibilità	Navigazione nel sistema	La pagina profilo non era raggiungibile tramite i menu di navigazione interna, ma solo tramite inserimento diretto dell'URL.	Isolamento logico del modulo profilo rispetto al framework e violazione dei principi di accessibilità.
Integrità dei dati	Comunicazione tra servizi	Mismatch nei tipi di dato: ID del giocatore in sessione era di tipo Long, mentre gli endpoint richiedevano parametri String.	Errori fatali di validazione e fallimento delle chiamate REST tra T5 e T23.
Persistenza	Database (T23)	Assenza o malfunzionamento del recupero dei dati di profilo dal database.	Visualizzazione di informazioni vuote o non aggiornate e perdita di affidabilità del sistema.
Dinamismo UI	Storico delle partite	Le partite recenti erano visualizzate tramite dati statici hardcoded nel front-end.	Impossibilità per l'utente di monitorare correttamente i propri progressi reali.
Design della logica	Front-end (T5)	Layout disordinato con elementi non coerenti al contesto (es. strumenti social nella vista match).	Violazione del principio di coerenza della UI e peggioramento dell'esperienza utente.
Modularità	Separazione dei livelli	Logica duplicata e ridondante distribuita tra template HTML e file JavaScript.	Codice difficilmente manutenibile ed alto rischio di inconsistenze logiche.
API	Interfaccia T5-T23	Naming degli endpoint e tipi di ritorno non coerenti tra front-end e back-end.	Difficoltà nell'integrazione di nuove funzionalità e aumento del debito tecnico.

Tabella 4 Problemi riscontrati

1.6.2 Casi d'uso

Analizzato il database e la pagina esistente, si ora studiano i vari casi d'uso possibili che vogliono implementare sulla pagina profilo.

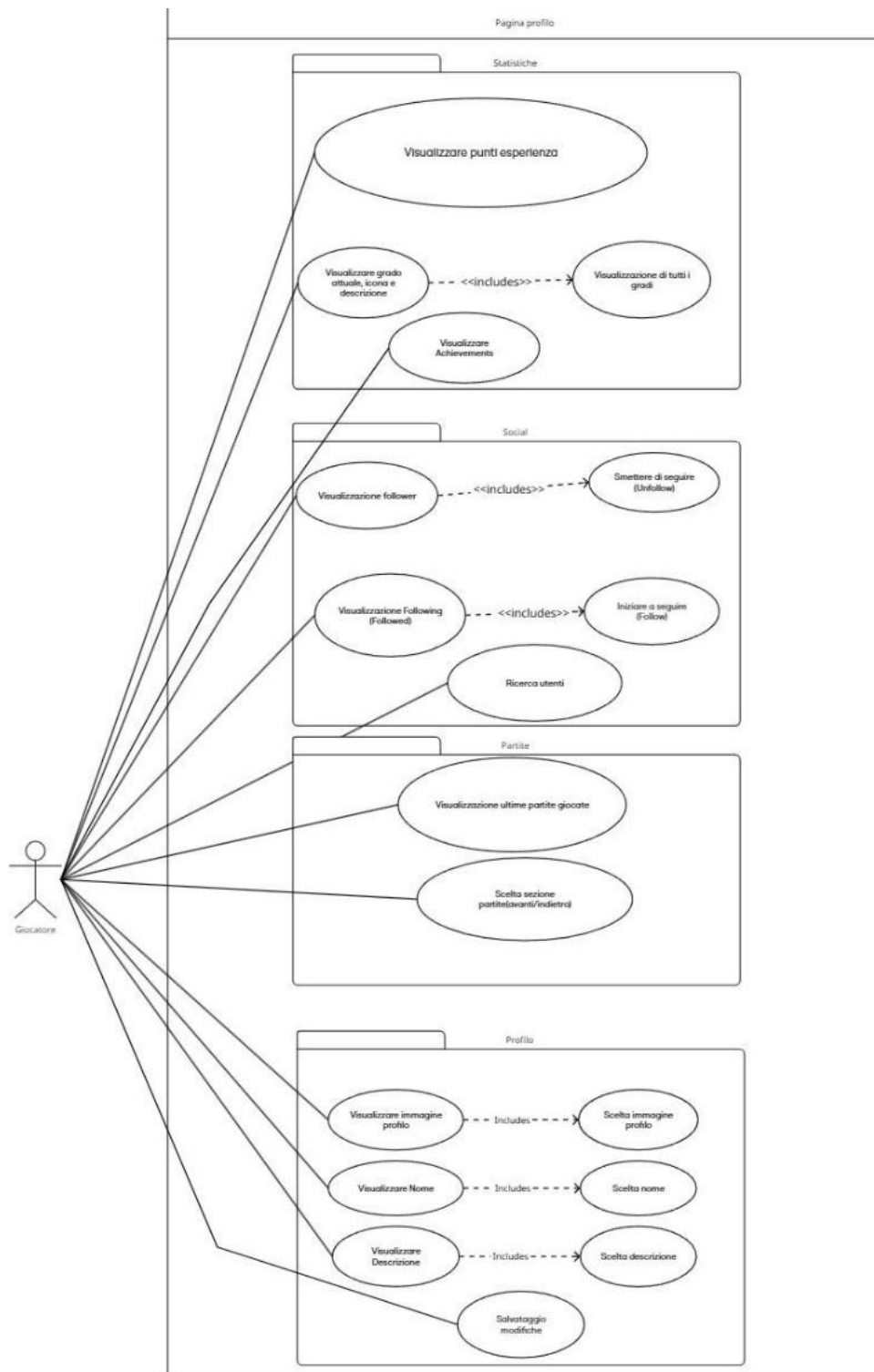


Figura 17 Casi d'uso

Nella presentazione del task veniva definito il selettore per scegliere la vista specifica che si voleva, si sono divisi quindi i casi d'uso in 4 macroaree o "moduli".

Modulo Statistiche

Il *modulo statistiche* è responsabile della gestione e della visualizzazione delle informazioni relative ai progressi del giocatore e ai risultati ottenuti durante l'esperienza di gioco. Esso rappresenta il fulcro del *sistema di gamification*, fornendo all'utente un feedback costante sul proprio avanzamento.

Visualizzazione dei punti esperienza (XP)

Consente al giocatore di monitorare il proprio avanzamento numerico all'interno del sistema, visualizzando i punti esperienza accumulati.

Visualizzazione del grado attuale

Permette di visualizzare il rango attualmente raggiunto dal giocatore, corredato da un'icona rappresentativa e da una descrizione esplicativa.

Relazione di inclusione (<<include>>), permette la visualizzazione di tutti i gradi ed include la possibilità di accedere a una vista informativa che illustra l'intera gerarchia dei livelli disponibili nel sistema di gamification.

Visualizzazione degli achievement

Consente al giocatore di consultare gli obiettivi sbloccati nel corso delle sessioni di gioco, sia a livello globale sia a livello di singola partita.

Modulo Social

Il *modulo social* gestisce le dinamiche relazionali della piattaforma, orchestrando le funzionalità di interazione tra gli utenti. Esso favorisce la costruzione di una community attiva e la scoperta di nuovi profili.

Visualizzazione dei Follower

Permette al giocatore di consultare l'elenco completo degli utenti che seguono il suo profilo.

Relazione (<<include>>): Segui (*Follow*) – Questa funzionalità consente di aggiungere un utente dalla propria lista following.

Visualizzazione dei Profili Seguiti

Mostra l'elenco degli utenti attualmente seguiti dal giocatore.

Relazione (<<include>>): Smetti di *seguire* (*Unfollow*) – Permette di eliminare una relazione di follow verso altri utenti.

Ricerca Utenti

Fornisce uno strumento di *query* che consente di individuare specifici giocatori tramite il loro nickname, fungendo da punto d'accesso primario per l'interazione sociale.

Modulo Partite

Il *modulo partite* è dedicato alla gestione e alla consultazione dello storico delle attività di gioco del giocatore, offrendo una visione dettagliata delle sessioni precedenti.

Visualizzazione delle ultime partite giocate

Consente di consultare le informazioni relative ai match precedenti, includendo l'esito della partita e i dettagli dell'avversario affrontato.

Modulo Profilo

Il *modulo profilo* gestisce i dati relativi all'identità digitale del giocatore e alle informazioni anagrafiche associate al profilo utente.

Visualizzazione e scelta dell'immagine del profilo

Permette la visualizzazione dell'avatar corrente e la selezione di una nuova immagine tra quelle predefinite.

Visualizzazione e modifica del nickname

Consente la visualizzazione del nome utente e la sua eventuale modifica.

Visualizzazione e modifica della descrizione

Permette la lettura e l'aggiornamento della biografia personale del giocatore.

Salvataggio delle modifiche

Si occupa della persistenza delle modifiche apportate (nickname, descrizione e avatar) nel database, mediante l'interazione con il microservizio T23.

CAPITOLO 2 Implementazione

2.1 Implementazione e struttura del progetto

Data la struttura di partenza, analizzata nel capitolo precedente, si è deciso di:

- Lasciare il *package model* di T23 invariato.
- Accorpare pagine Achievements, Profile-Edit e Profile insieme e renderle modulari.
- Definire un prototipo UI.

Per cui si è partiti da un template di riferimento che mostra le varie funzionalità e come queste vengono implementate per i vari casi d'uso.

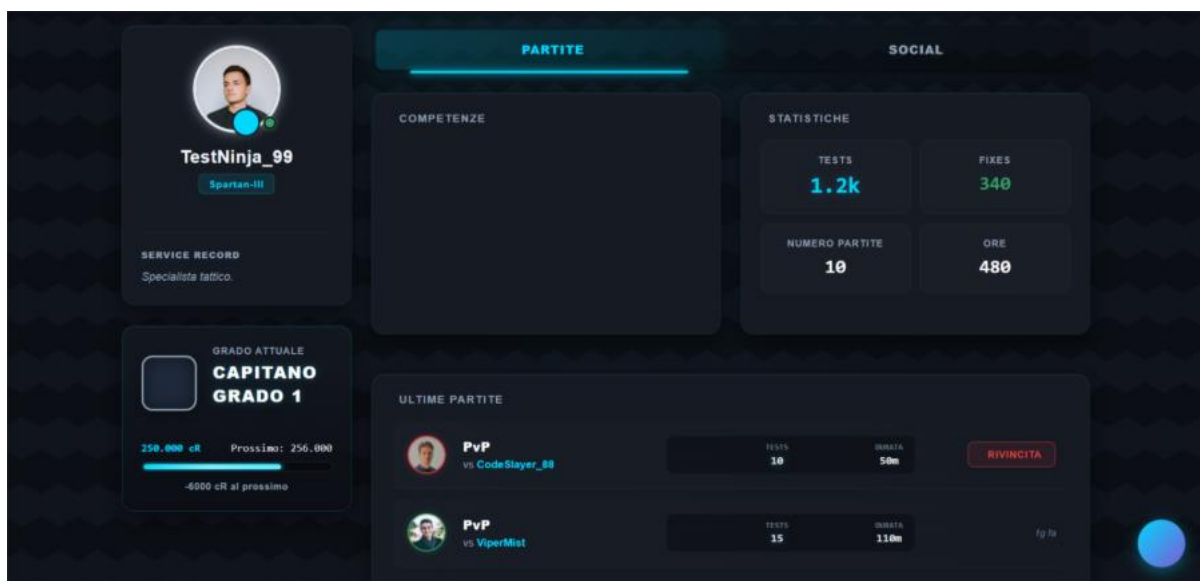


Figura 18 Prototipo della vista iniziale profilo

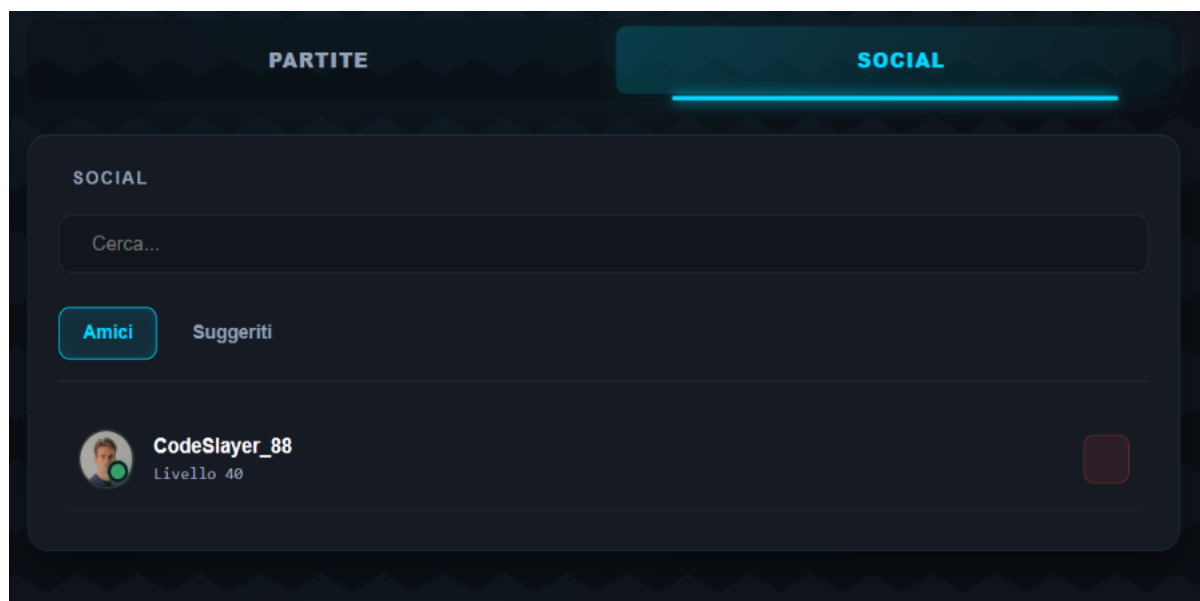


Figura 19 Prototipo della vista social

2.1.1 Comunicazione T5-T23

Per mantenere il pattern MVC, si è adottato il seguente flusso dal giocatore al database in T23 per tutte le funzioni implementate:

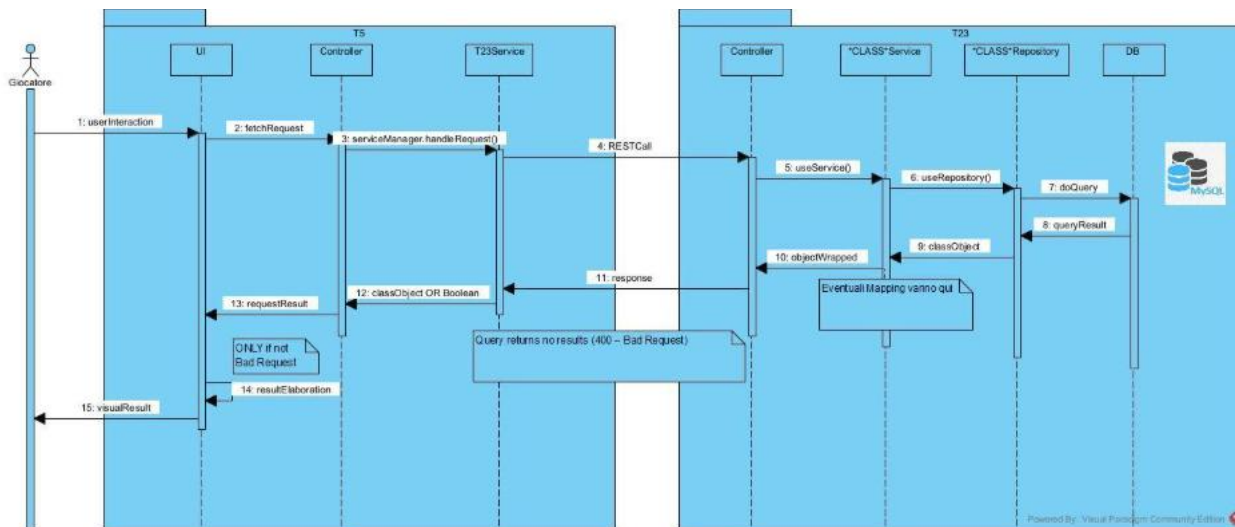


Figura 20 Iterazione tra container

Il *Sequence Diagram* mostra come un'interazione dell'utente sulla UI di T5 scateni una chiamata fetch verso l'*UserProfileController*. Quest'ultimo dialoga con il back-end T23, il quale, tramite il pattern *Service-Repository*, interroga il database MySQL. Il ritorno dei dati avviene tramite chiamate RESTful, garantendo un payload ottimizzato e riducendo il disaccoppiamento tra i modelli dei due microservizi.

2.2 Implementazione di alcuni casi d'uso

2.2.1 Logica di controllo: gestione accesso e fall-back

Il primo scenario operativo analizzato riguarda l'inizializzazione della pagina profilo. L'activity diagram descrive il processo di validazione del JWT Token per garantire la sicurezza dell'accesso. Una scelta implementativa di rilievo riguarda la gestione dei dati mancanti: il sistema verifica la presenza del profilo nel database di T23, in caso di assenza o di primo accesso attiva una logica di fall-back che calcola statistiche iniziali neutre, garantendo che l'utente visualizzi sempre un'interfaccia coerente e priva di errori di rendering.

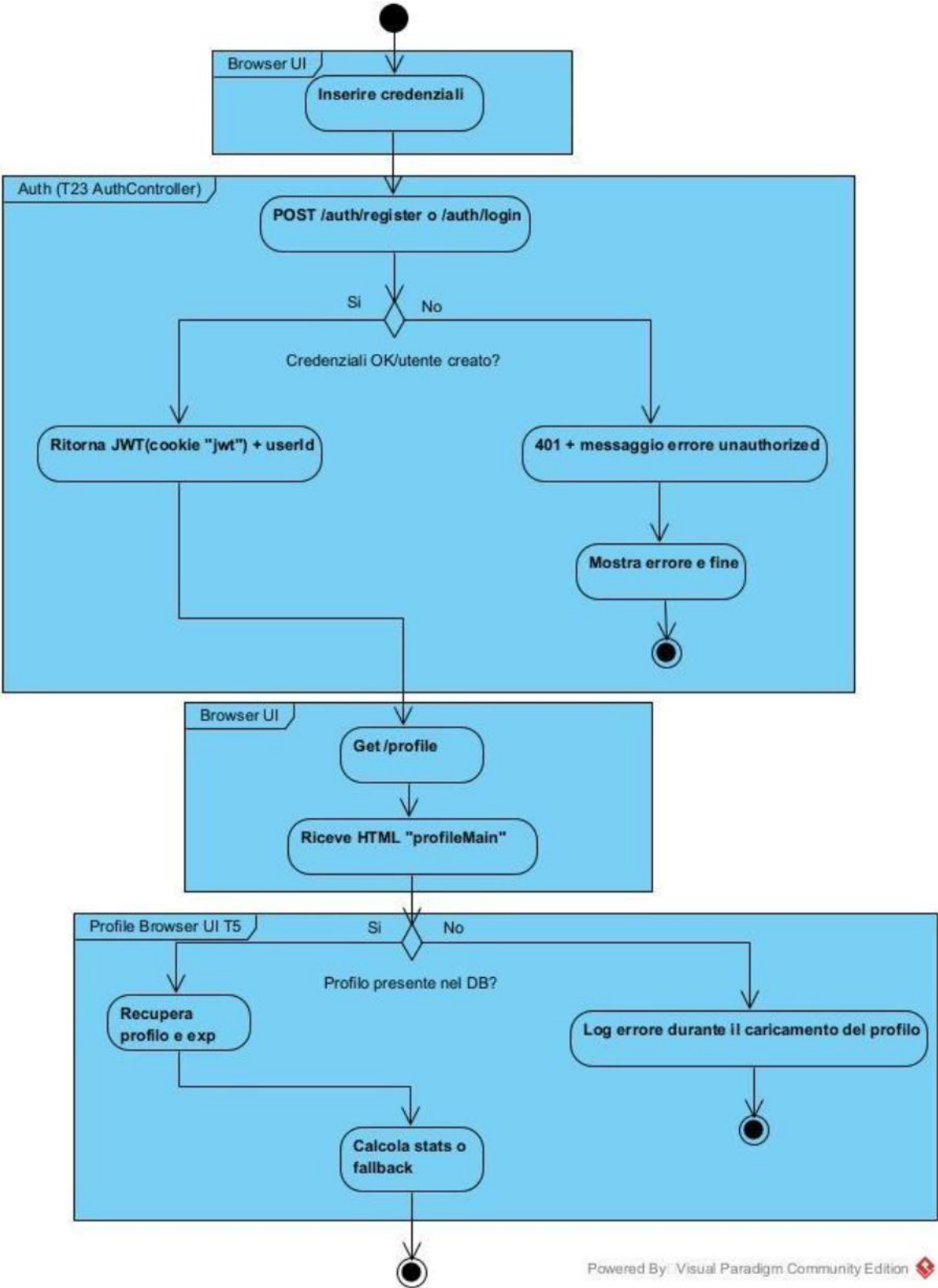


Figura 21 Inizializzazione del profilo

Di seguito vengono illustrate tutte le viste incontrate sul percorso dopo aver fatto il login o la registrazione correttamente:



Figura 22 Schermata main con autenticazione

Dopo aver premuto sulla propria email in alto a destra verrà caricata la pagina con i propri dati (se presenti nel database).

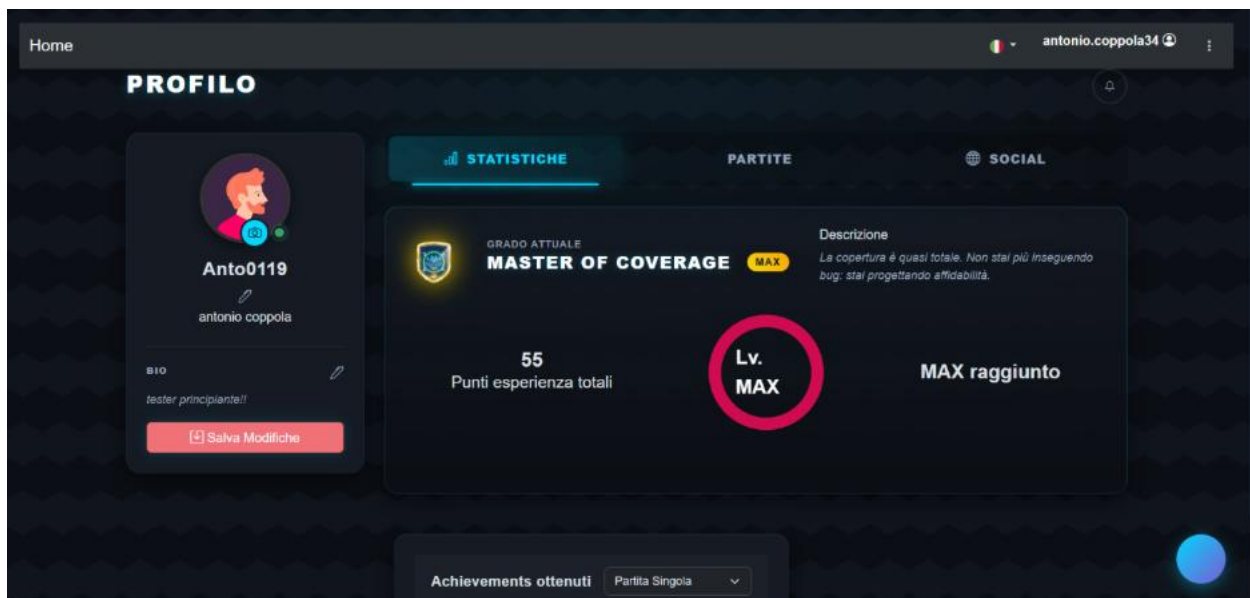


Figura 23 Pagina profilo main di un giocatore

Sulla barra di navigazione (navbar), si ha possibilità di cambiare la lingua (non ancora implementato sulla pagina), dellogare o tornare alla Home.

2.2.2 Dinamica social: aggiornamento asincrono dello stato

Uno dei requisiti più complessi (RF8) ha riguardato la gestione del grafo sociale senza ricaricare la pagina. L'activity diagram del toggle follow illustra la logica di business implementata: quando l'utente decide di smettere di seguire un profilo, il sistema invia una richiesta POST asincrona. Solo a seguito della conferma di avvenuta persistenza, nel database di T23 il front-end aggiorna lo stato locale della lista *following*, rimuovendo l'elemento visivamente. Questo garantisce una sincronizzazione perfetta tra la vista e lo stato reale dei dati nel back-end.

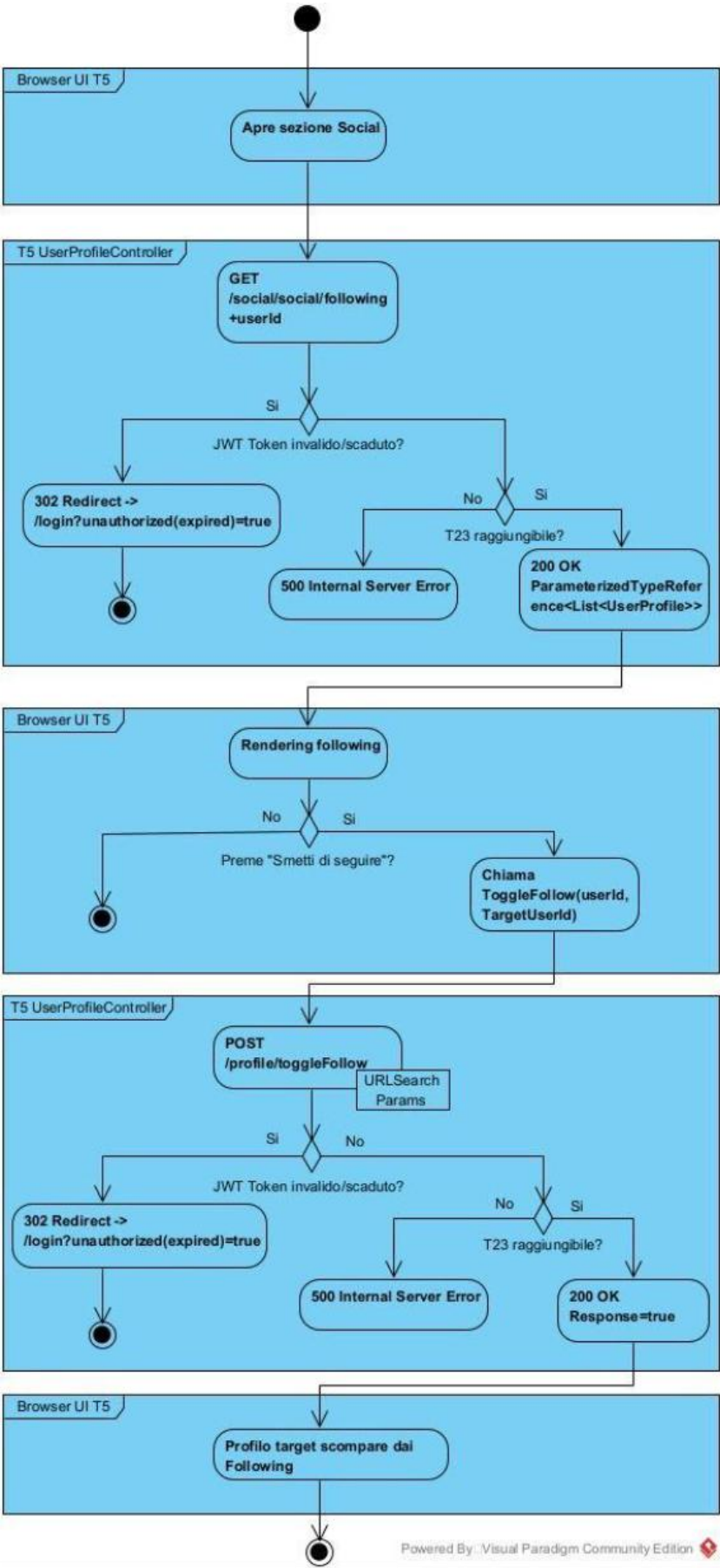


Figura 24 Activity diagram del toggle follow

Di seguito le diverse possibili viste di questo percorso (follow/unfollow):

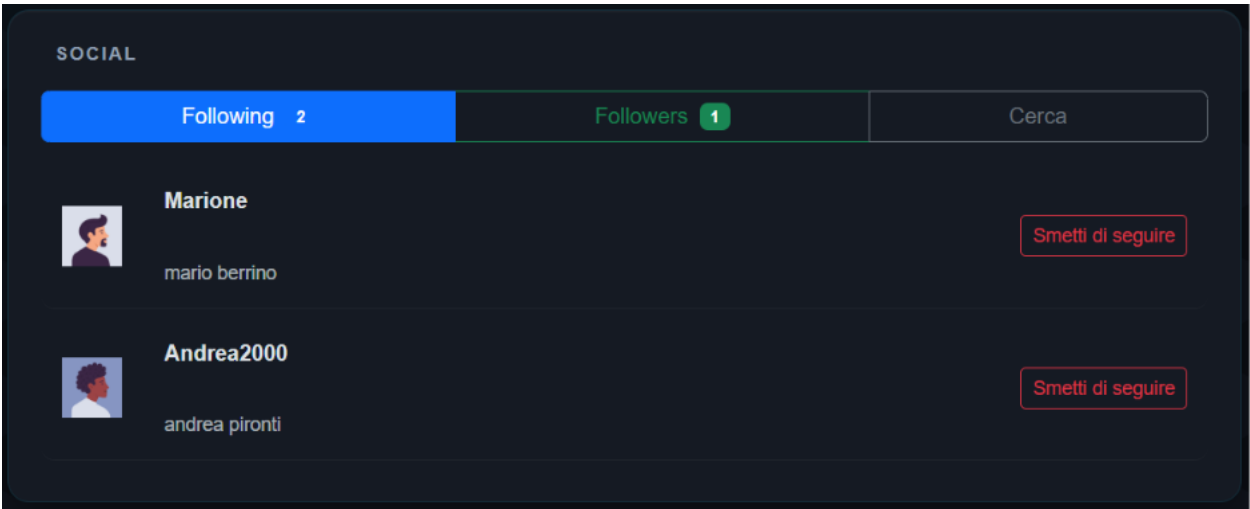


Figura 25 Vista elenco dei following

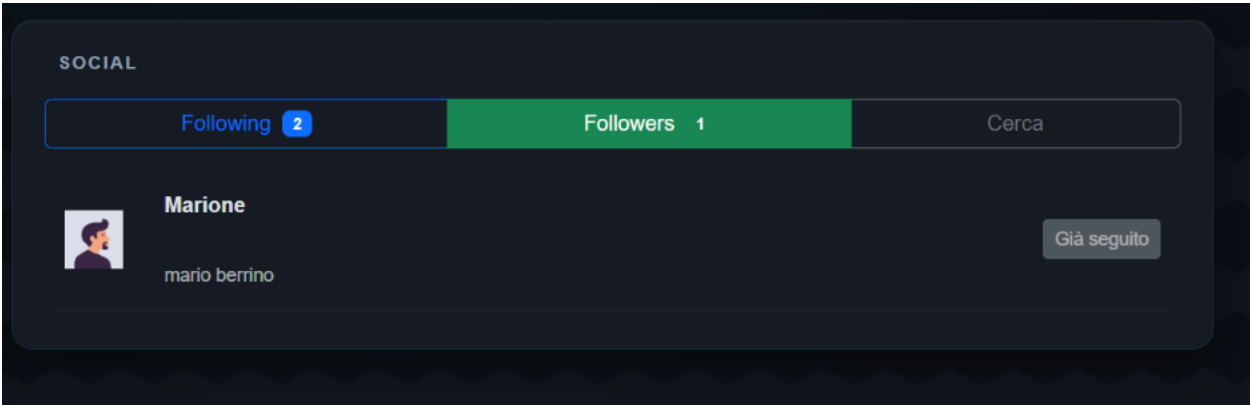


Figura 26 Vista dei follow già seguiti

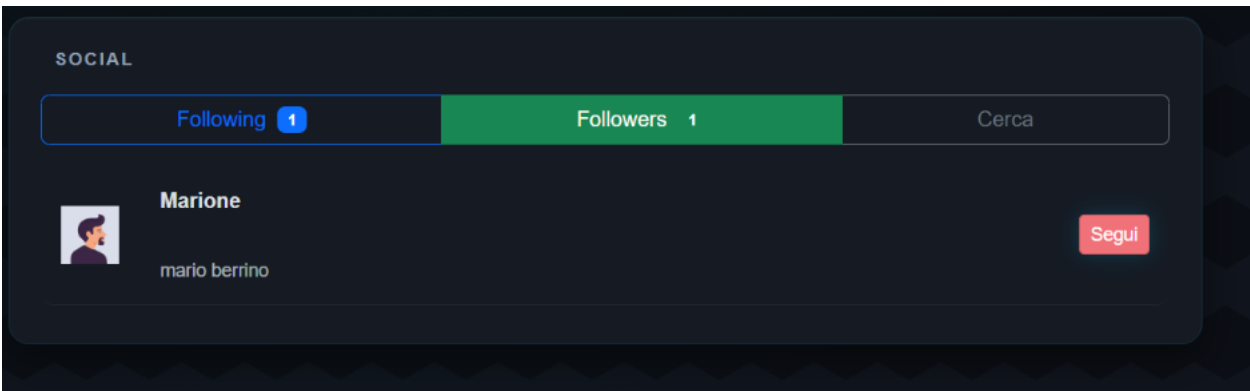


Figura 27 Vista follow non seguiti

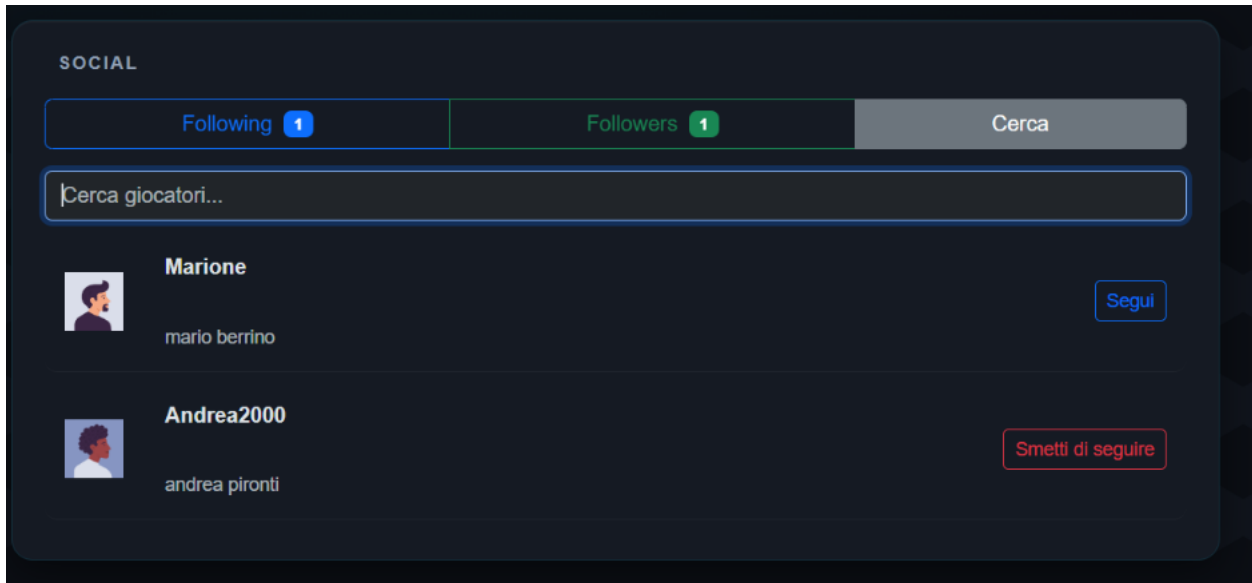


Figura 28 Vista della ricerca dei giocatori

2.2.3 Logica di editing profilo: diagramma delle attività

Una volta stabilita la struttura modulare del front-end, è stato necessario definire con precisione i flussi di controllo che regolano la modifica dei dati sensibili, come l'avatar, il nickname e la biografia del giocatore (RF1, RF4). Come illustrato nel diagramma delle attività riportato di seguito, la progettazione si è concentrata sulla realizzazione di un sistema di validazione a due livelli per garantire l'integrità dei dati e l'efficienza delle risorse di rete.

Il flusso operativo ha inizio nel browser dell'utente, all'interno della UI di T5, dove il modulo `profileFixed.js` intercetta ogni tentativo di modifica. In questa fase, prima ancora che la richiesta lasci il client, il sistema esegue un controllo rigoroso sui limiti di caratteri stabiliti (20 per il nickname e 200 per la bio). Questa scelta di design permette di fornire un feedback immediato tramite alert in caso di input non conformi, bloccando preventivamente la propagazione di errori verso il server e riducendo il carico inutile sui microservizi (RNF5).

Solo dopo aver superato la validazione locale, la logica di business procede verso lo strato di back-end tramite una chiamata asincrona POST `/profile/save` indirizzata al *UserProfileController*. Durante questa fase, il sistema esegue un controllo critico sulla sicurezza verificando la validità e la scadenza del JWT Token. Per cui se l'autenticazione ha successo e la comunicazione con il microservizio T23 è stabile, le modifiche vengono rese persistenti nel database MySQL, restituendo all'utente una conferma dell'operazione e garantendo la perfetta sincronizzazione tra l'identità digitale visualizzata e i dati memorizzati nel sistema.

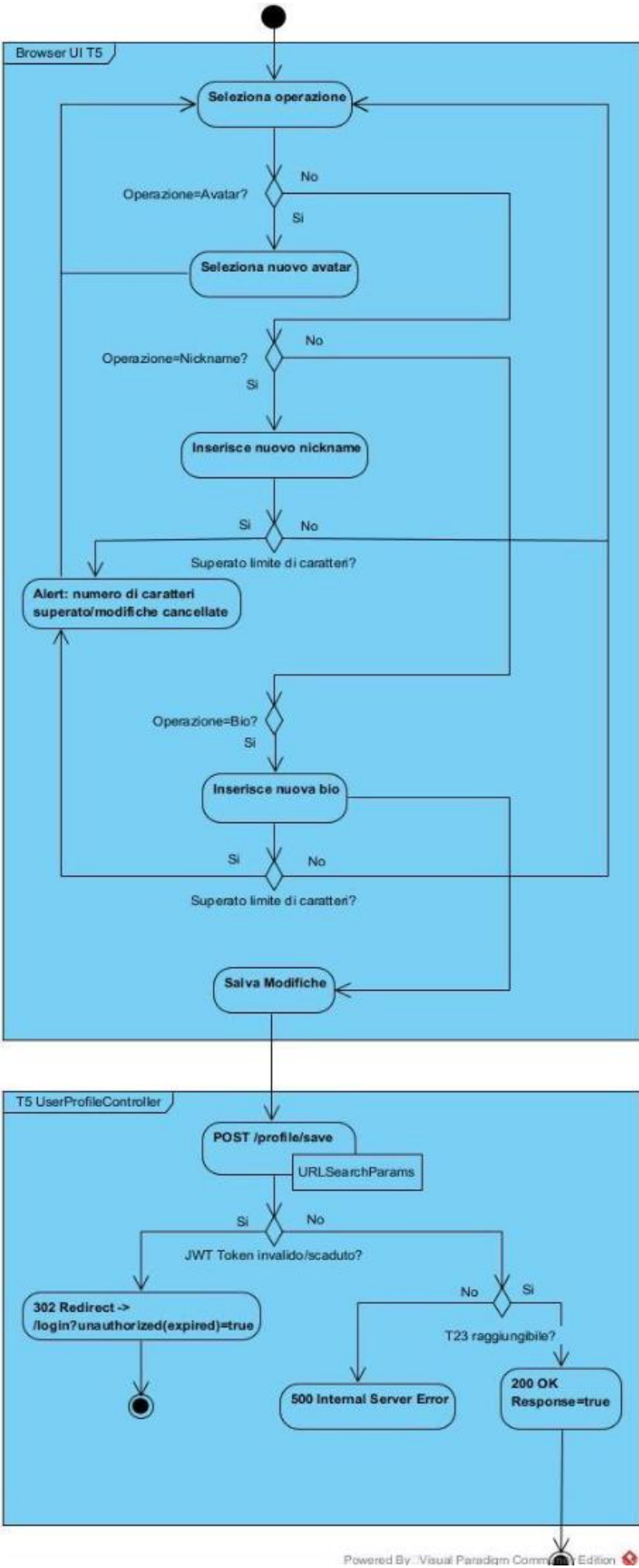


Figura 29 Activity diagram della modifica e salvataggio profilo

Di seguito si può vedere come viene presentata questa immagine nel prodotto finale:



Figura 30 Vista di default profilo

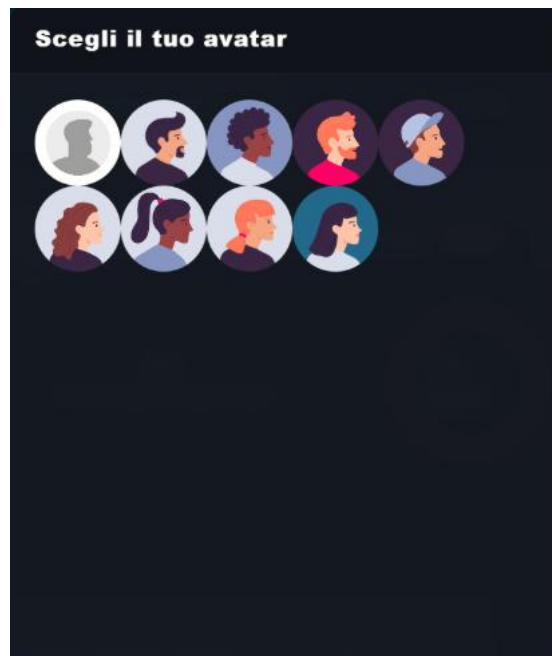


Figura 31 Modale per la scelta dell'avatar (standard)

Per adesso si può scegliere solo tra gli avatar offerti dal servizio, per uscire dal modale basta premere su qualsiasi punto dello schermo, se si preme su un'immagine essa verrà visualizzata sulla pagina ma non ancora salvata nel database.

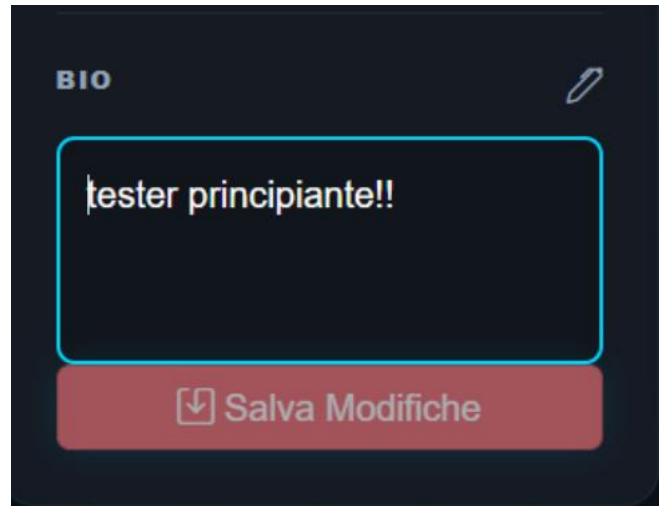


Figura 32 Schermata modifica della bio

Mentre si modifica una qualsiasi delle parti del profilo, il pulsante “Salva Modifiche” viene disattivato per permettere la sincronizzazione dei dati. Questo funzionamento può essere descritto da macchina a stati.

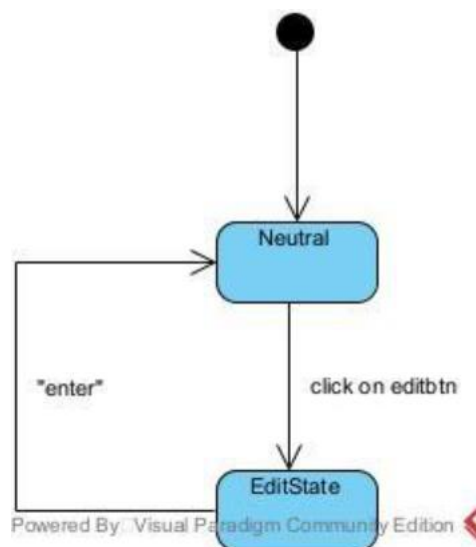


Figura 33 State machine diagram per l'editing del profilo

2.2.4 Vista dati: modellazione della persistenza per lo storico partite

Dopo aver analizzato la logica di validazione e i flussi di controllo, è fondamentale soffermarsi su come queste operazioni si riflettano concretamente sulla base dei dati. La transizione da un sistema basato su dati statici a una piattaforma dinamica ha richiesto infatti, un ripensamento delle relazioni tra le entità coinvolte nella gestione dei progressi.

Come illustrato nel diagramma sottostante, Figura 34, l'attenzione si è concentrata sulla creazione di un legame solido e trasparente tra l'identità del giocatore e le sue performance sul campo.

In questa nuova configurazione, la tabella dedicata allo storico delle sessioni non è più un elemento isolato ma diventa il punto di convergenza tra l'anagrafica dell'utente e i dati tecnici della sfida, come il tipo di avversario affrontato e il livello di difficoltà impostato. Questo approccio permette al microservizio T23 di interrogare il database in modo estremamente mirato: grazie a una struttura basata su chiavi esterne ben definite, il sistema è ora in grado di ricostruire l'intera esperienza di gioco (inclusi gli achievement ottenuti nella singola partita) e di servirla al front-end tramite un payload ottimizzato. Tale modellazione non solo garantisce l'integrità referenziale del sistema, ma abilita quella fluidità di visualizzazione richiesta dai nuovi obiettivi di progetto, assicurando che ogni dato mostrato sul profilo sia il risultato di un'effettiva interazione salvata in modo persistente sul database.

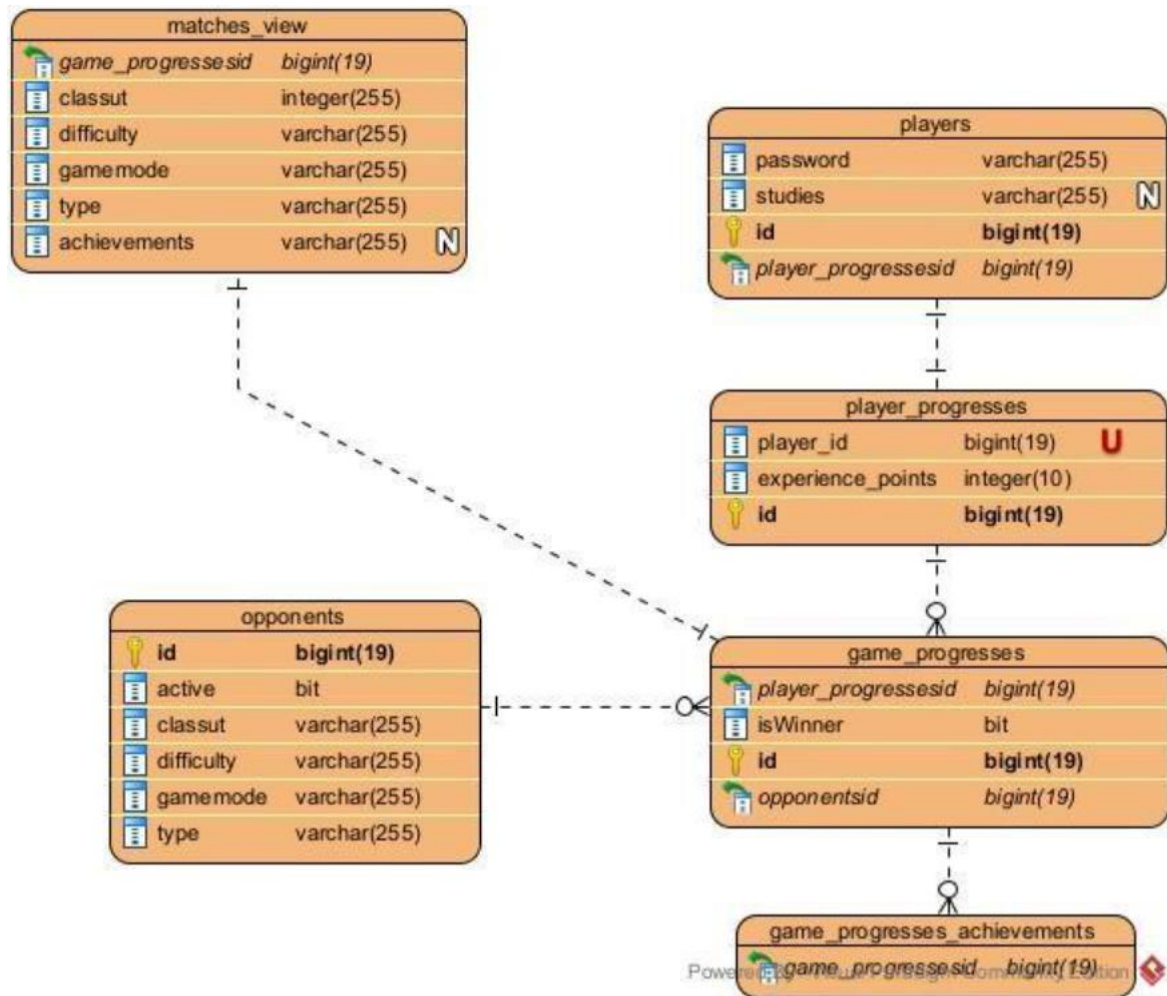


Figura 34 GameProgressVisibility

Il diagramma in esame illustra lo schema relazionale del database MySQL gestito internamente dal microservizio T23, focalizzandosi sulla persistenza e sulla visibilità dei progressi di gioco. Questo centralizza qui tutte le informazioni necessarie a ricostruire l'attività del giocatore. La struttura si muove attorno alla tabella *game_progresses*, che funge da connettore tra l'identità dell'utente e i dettagli tecnici di ogni sfida, permettendo di recuperare in modo mirato l'esito dei vari match, i robot avversari affrontati e gli achievement ottenuti nella singola sessione. Per rendere questi dati fruibili dal front-end (T5) senza esporre la complessità delle tabelle fisiche, si è implementato un Data Transfer Object (DTO) denominato *GameProgressDTO*. Questo oggetto funge da contenitore ottimizzato che trasporta verso l'interfaccia utente solo le informazioni strettamente necessarie al rendering della vista "Matches", garantendo al contempo la sicurezza dei dati sensibili e il disaccoppiamento tra i microservizi. In questo modo, ogni riga visualizzata nello storico del profilo non è più un semplice valore statico ma il risultato di una mappatura dinamica di entità reali salvate nel database.

Di seguito come viene visualizzata l'aggregazione di questi dati:

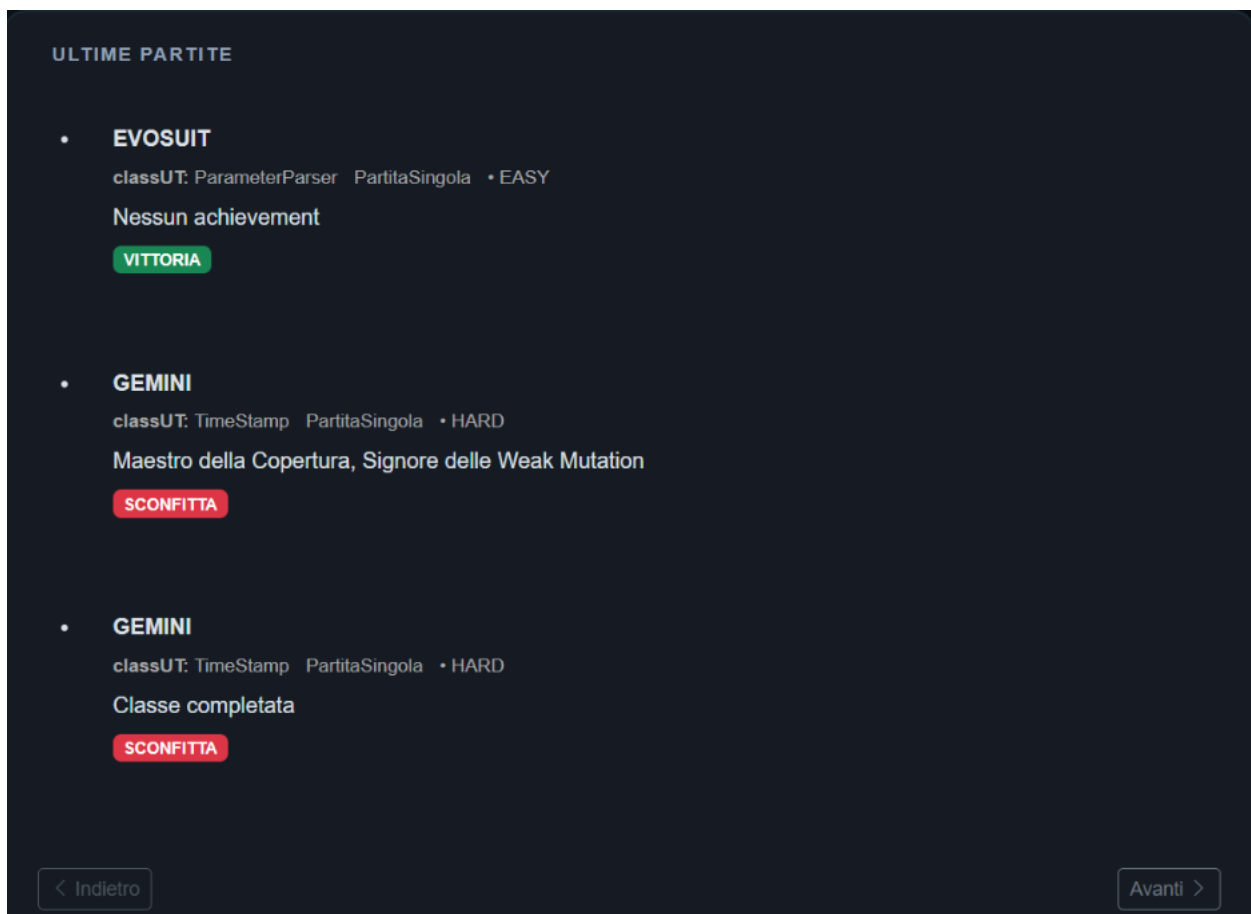


Figura 35 Vista dello storico partite

2.3 Descrizione delle decisioni di progetto

In conclusione queste sono le decisioni prese per implementare il task assegnato:

Componente	Decisione di progetto	Requisiti soddisfatti	Esempio
Front-tend (T5): Architettura JavaScript	Implementazione di una struttura a moduli asincroni (<code>profileMain.js</code> , <code>stats.js</code> , <code>social.js</code> , ecc.) per gestire dinamicamente le viste senza ricaricamento.	RF2, RF3, RF5, RF6, RF7, RF8, RNF5	Creazione del file <code>profileMain.js</code> che agisce da orchestratore, utilizzando <code>import</code> per caricare moduli come <code>stats.js</code> e <code>social.js</code> .
Front-end (T5): UI Refactoring	Sostituzione delle pagine statiche (<code>Achievements.html</code> , <code>GameHistory.html</code> , <code>Profile-Edit.html</code>) con un layout unico basato su sezioni modulari (tab).	RF2, RNF2, RNF3	Integrazione dei contenuti di <code>Achievements.html</code> e <code>GameHistory.html</code> all'interno di un'unica pagina <code>profile.html</code> gestita via script.
Back-end (T23): Business Logic	Introduzione di <code>GameProgressService</code> per centralizzare la gestione dello storico partite.	RF7, RF8, RNF4	Implementazione di <code>GameProgressService</code> in T23 per il recupero dello storico tramite il metodo <code>getGameHistoryByPlayer</code> .
Interfaccia (T5-T23): Comunicazione	Definizione di nuove rotte API dedicate nel <code>UserProfileController</code> per ottimizzare il payload.	RF4, RF7, RNF5	Utilizzo di <code>GameProgressDTO</code> per trasmettere i dati dei match da T23 a T5, evitando l'esposizione delle entità di database.

Back-end (T23): Validazione Dati	Introduzione di vincoli di integrità applicativa (es. limiti caratteri per bio e nickname) prima della persistenza su MySQL.	RF4, RNF4	Introduzione di controlli di lunghezza massima sui campi nickname (20) e bio (200) nel backend prima della persistenza.
Frontend (T5): Editing In-place	Sostituzione della pagina <code>Profile-Edit.html</code> con una logica di modifica integrata (<code>profileFixed.js</code>) per migliorare la fluidità d'uso.	RF1, RF4, RNF2	Modulo <code>profileFixed.js</code> che gestisce i campi input per bio e nickname direttamente nella vista profilo principale.

Tabella 5 Decisioni da implementare

2.4 Nuova organizzazione degli JavaScript per il profilo

La decisione progettuale cardine è stata la scomposizione del front-end nel microservizio T5 attraverso un approccio a componenti JavaScript.

Il file `profileMain.js` funge da orchestratore centrale, successivamente creato dal file originale **profile.js**, il quale durante la stesura del codice, difficile nella lettura e nella gestione dell'implementazione si è suddiviso in:

- **stats.js** e **matches.js**: gestiscono il recupero e il rendering dei dati di gioco.
- **social.js**: incapsula la logica del grafo sociale.
- **profileFixed.js**: controlla l'anagrafica e l'editing in-place.

Questa struttura garantisce l'estensibilità del sistema (RNF1) e una netta separazione delle varie responsabilità.

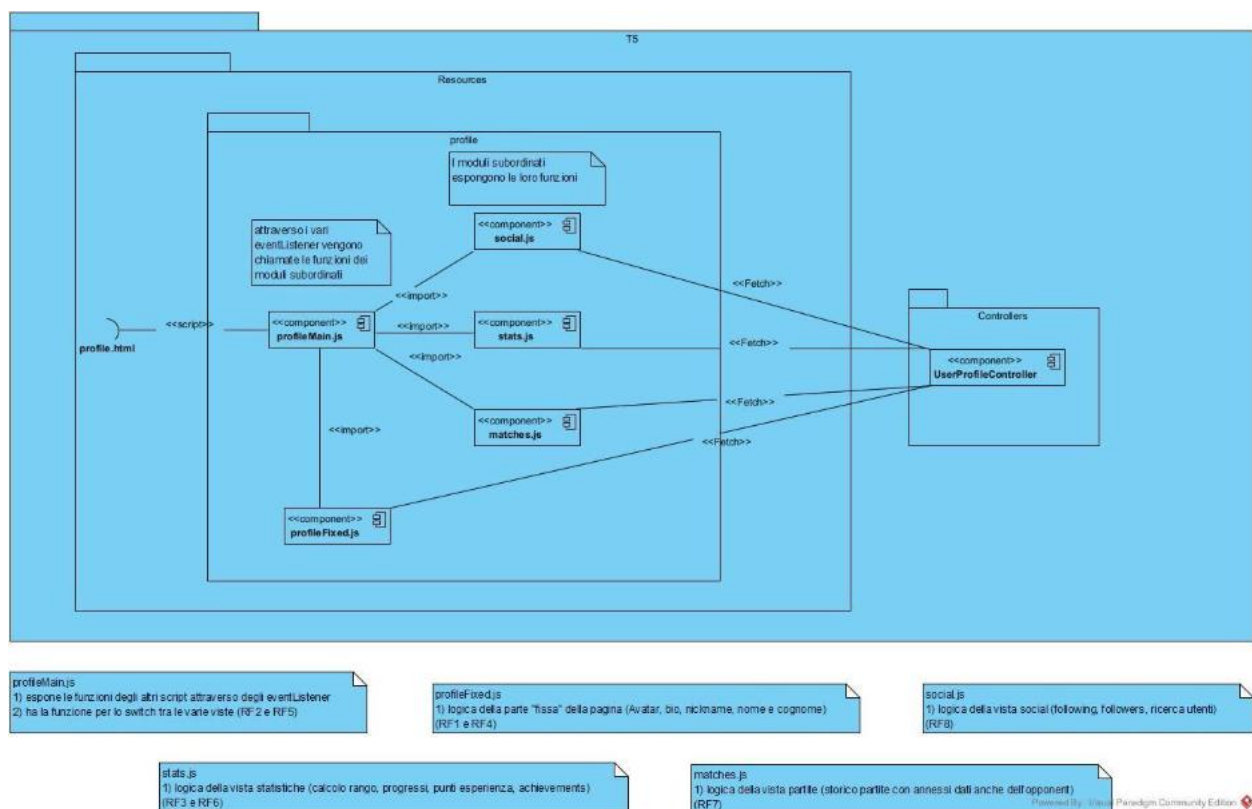


Figura 36 Microservizio T5

Nel lato client, le risorse sono state organizzate per supportare l'architettura a componenti JavaScript introdotta con il Task R6.

All'interno della directory `src/main/resources/static/t5/js`, è stata creata la sottocartella dedicata `profile`, che ospita i moduli descritti nel package diagram nella figura 36, tra cui `profileMain.js` e i suoi script subordinati. Tale separazione fisica dei file rispecchia la separazione logica delle responsabilità, assicurando che le modifiche apportate alla vista social non interferiscano con la logica delle statistiche o dei match.

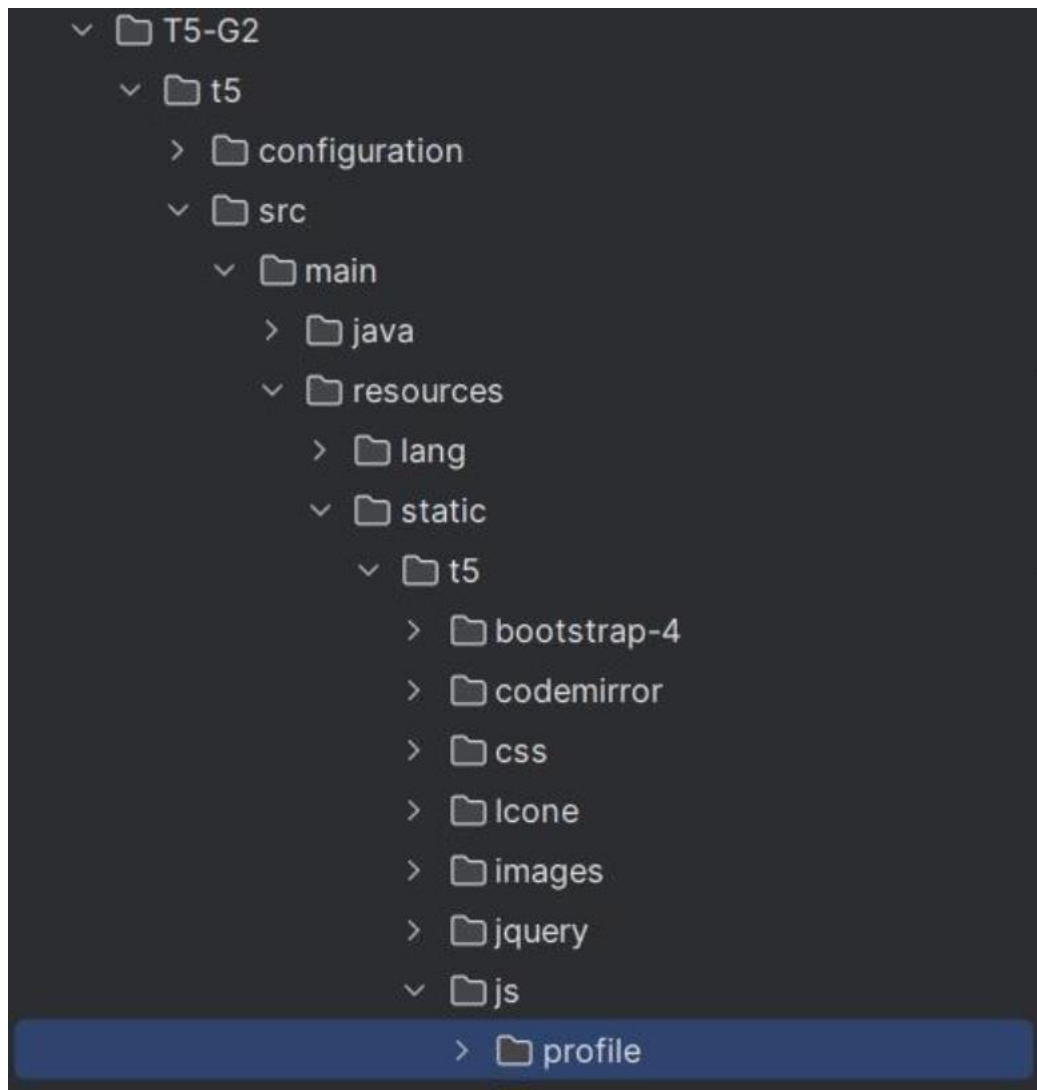


Figura 37 T5, resources

2.5 Modifiche apportate ai file

Di seguito vengono riassunte tutte le aggiunte, modifiche ed eliminazioni fatte in vari punti del codice.

Categoria	Modulo	Modifica
Aggiunti (Front-end)	profileMain.js	Punto di ingresso principale: gestisce l'inizializzazione dei listener e lo switch dinamico delle view (Social, Matches, Stats) senza ricaricare la pagina.
Aggiunti (Front-end)	profileFixed.js	implementa la logica di editing per Bio e Nickname con limiti di caratteri (20/200) e la selezione dell'avatar.
Aggiunti (Front-end)	stats.js	gestisce la mappatura degli XP nei dieci Ranghi, l'aggiornamento della barra di progresso e il rendering degli achievement.
Aggiunti (Front-end)	social.js	gestisce le chiamate per follower/following, la logica di ricerca utenti e l'aggiornamento dello stato del follow.
Aggiunti (Front-end)	matches.js	recupera la storia dei match, gestendo la paginazione lato client e la visualizzazione dei badge vittoria/sconfitta.
Aggiunti (Back-end)	GameProgressService (T23)	implementa la logica per il recupero dello storico completo delle partite di un giocatore tramite getGameHistoryByPlayer. Utilizza il MapperFacade per restituire una lista di GameProgressDTO al microservizio T5.
Modificati (Back-end)	UserProfileController (T5)	aggiunte rotte utili per la logica della pagina profilo
Modificati (Back-end)	T23Service (T5)	aggiunte servizi utili per la logica della pagina profilo
Modificati (Back-end)	UserGameProgress (T5)	aggiunto attributo "Achievements"
Modificati (Back-end)	UserSocialController (T23)	aggiunte rotte per editare profilo e cercare profili utenti tramite termini di ricerca
Modificati (Back-end)	UserSocialService (T23)	aggiunti servizi per editare profilo e cercare profili utenti tramite termini di ricerca

Rimossi dalla logica ma non dal progetto (Front-end)	Achievements.html (T5)	Vecchia pagina degli obiettivi; i contenuti sono ora integrati dinamicamente nel nuovo profilo.
Rimossi dalla logica ma non dal progetto (Front-end)	Profile-Edit.html (T5)	Vecchia pagina di modifica; sostituita dalla modalità di editing "in-place" gestita da profileFixed.js
Rimossi dalla logica ma non dal progetto (Front-end)	GameHistory.html (T5)	Vecchia pagina dello storico partite; i contenuti sono ora integrati dinamicamente nel nuovo profilo.
Rimossi dalla logica ma non dal progetto (Back-end)	UserProfileController (T5)	rotta per /Achievements deprecata.
Rimossi dalla logica ma non dal progetto (Back-end)	UserProfileController (T5)	rotta per /Edit-Profile deprecata.
Rimossi dalla logica ma non dal progetto (Back-end)	UserProfileController (T5)	rotta per /GameHistory deprecata.
Da implementare (sviluppi futuri)	Notifiche Profilo	Da implementare le notifiche sulla pagine /profile.
Da implementare (sviluppi futuri)	Immagini Personalizzate	La scelta degli avatar è limitata a una lista fissa; l'upload di file personalizzati è rimandato.
Da implementare (sviluppi futuri)	Chat tra utenti	Chat tra utenti che si seguono ancora da implementare.

Tabella 6 Modifiche apportate al progetto

2.6 GameProgressService

GameProgressService, aggiunto nella cartella service di T23, serve alla gestione della classe *GameProgress* che non presentava un servizio dedicato.

Da notare che i dati specifici per le partite sono gestiti da T4, in cui è presente una classe dedicata ai games ma, lavorando sul profilo giocatore specificatamente, si è preferito utilizzare la classe *GameProgress* T23 più dedicata agli achievements e agli opponents.

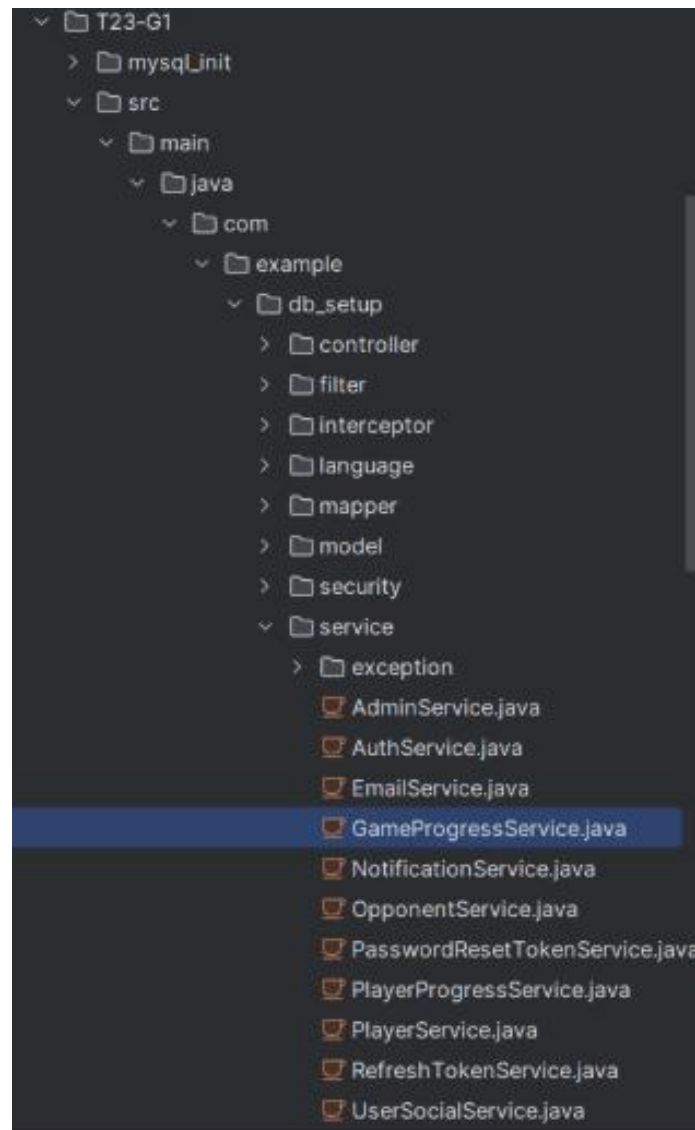


Figura 38 Ubicazione di *GameProgressService.java*

```
/**
 * Recupera lo storico completo delle partite di un player
 */
public List<GameProgressDTO> getGameHistoryByPlayer(Long playerId) {
    logger.info("Recupero storico partite per playerId={}", playerId);

    if (playerId == null) {
        throw new IllegalArgumentException("playerId nullo");
    }

    List<GameProgress> entities = gameProgressRepository.findGameProgressByPlayer(playerId);
    logger.info("Entità recuperate: {}", entities);

    List<GameProgressDTO> results = entities.stream()
        .map(mapperFacade::toDTO)
        .toList();

    logger.info("Results DTO: {}", results);

    return results;
}
```

Figura 39 Funzione di GameProgressService

Il servizio è stato implementato in modo da seguire il pattern utilizzato da **Spring Framework Service-Repository**.

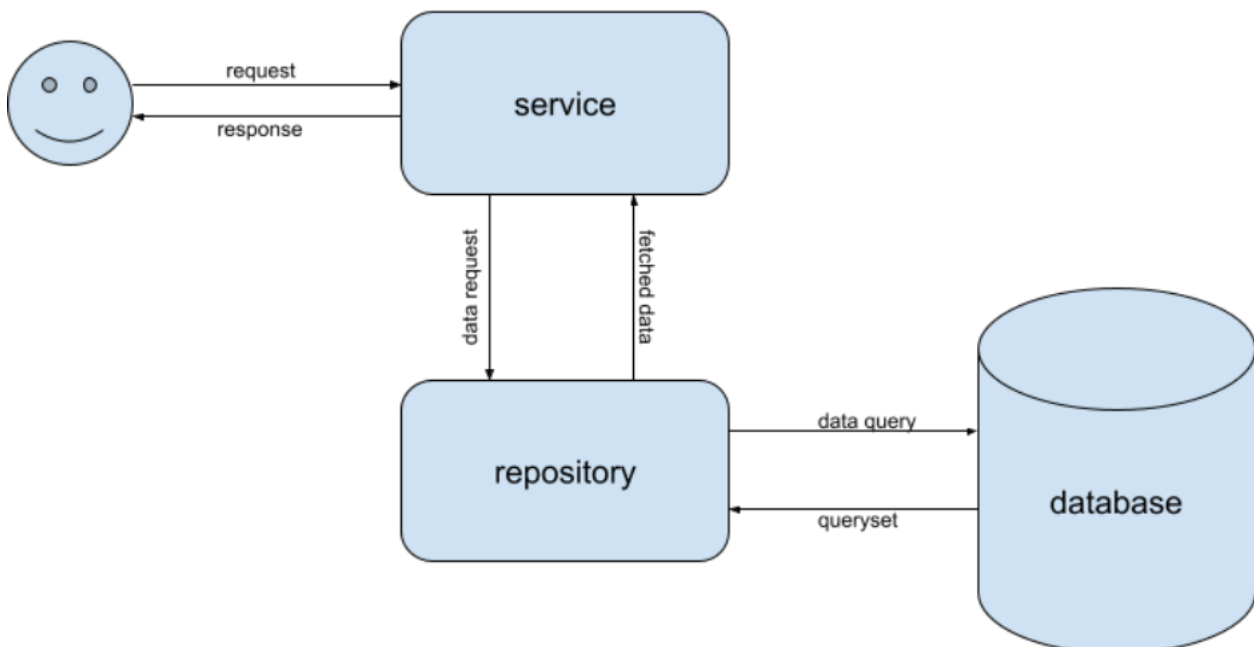


Figura 40 Diagramma del pattern Service-Repository

CAPITOLO 3 Testing

3.1 Testing e validazione

Alla fine di ogni implementazione delle varie funzionalità si è effettuato testing direttamente sulla pagina, analizzando il comportamento del codice end-to-end. Si è poi verificato se tutti i requisiti funzionali e non funzionali fossero stati risolti.

3.1.1 Test dei requisiti funzionali:

- **RF1 (visualizzazione informazioni fisse)**

È stato verificato che all'apertura della pagina il sistema recuperi correttamente l'avatar, il nickname e la biografia del giocatore. Il layout risponde correttamente popolando la sidebar sinistra con i dati statici previsti.

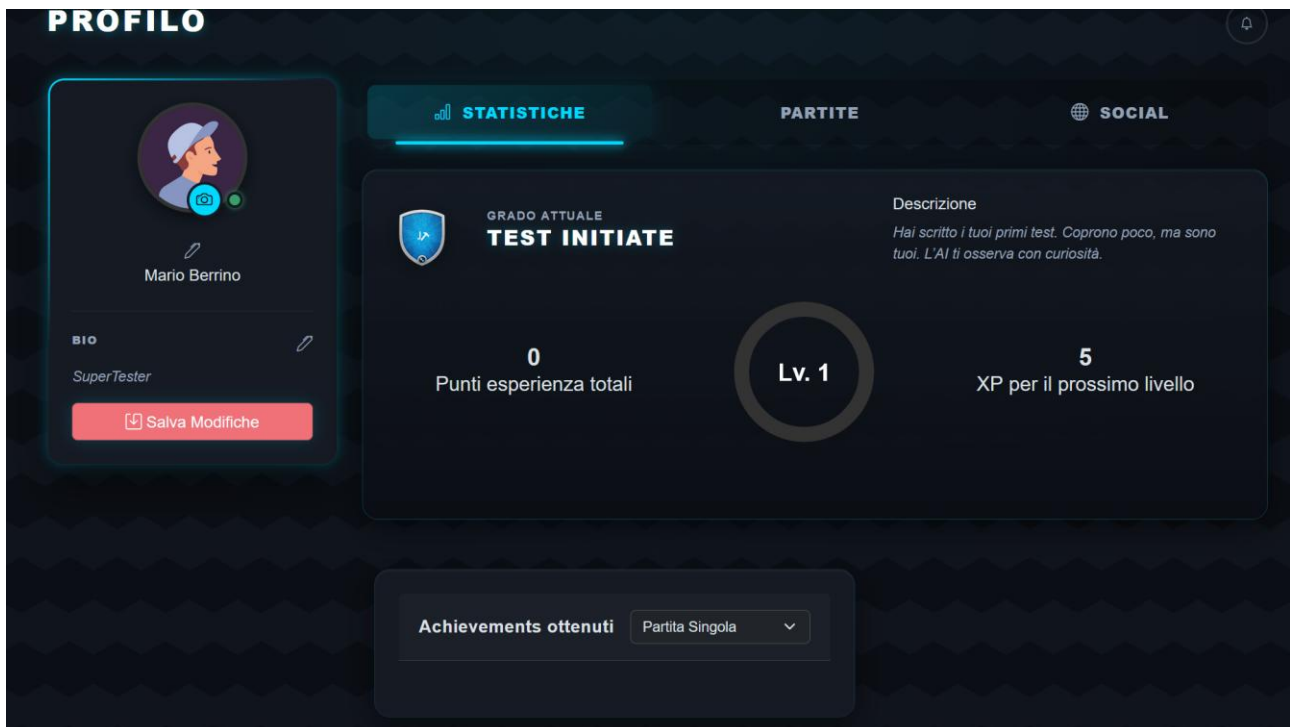


Figura 41 Correttezza caricamento pagina profilo

- **RF2 e RF3 (selezione contenuti dinamici e achievement)**
Attraverso l'interazione con il selettore "Achievements ottenuti", è stata confermata la capacità del sistema di filtrare e mostrare dinamicamente le informazioni (es. "Partita Singola"). La transizione tra le diverse categorie avviene istantaneamente e senza errori di rendering.

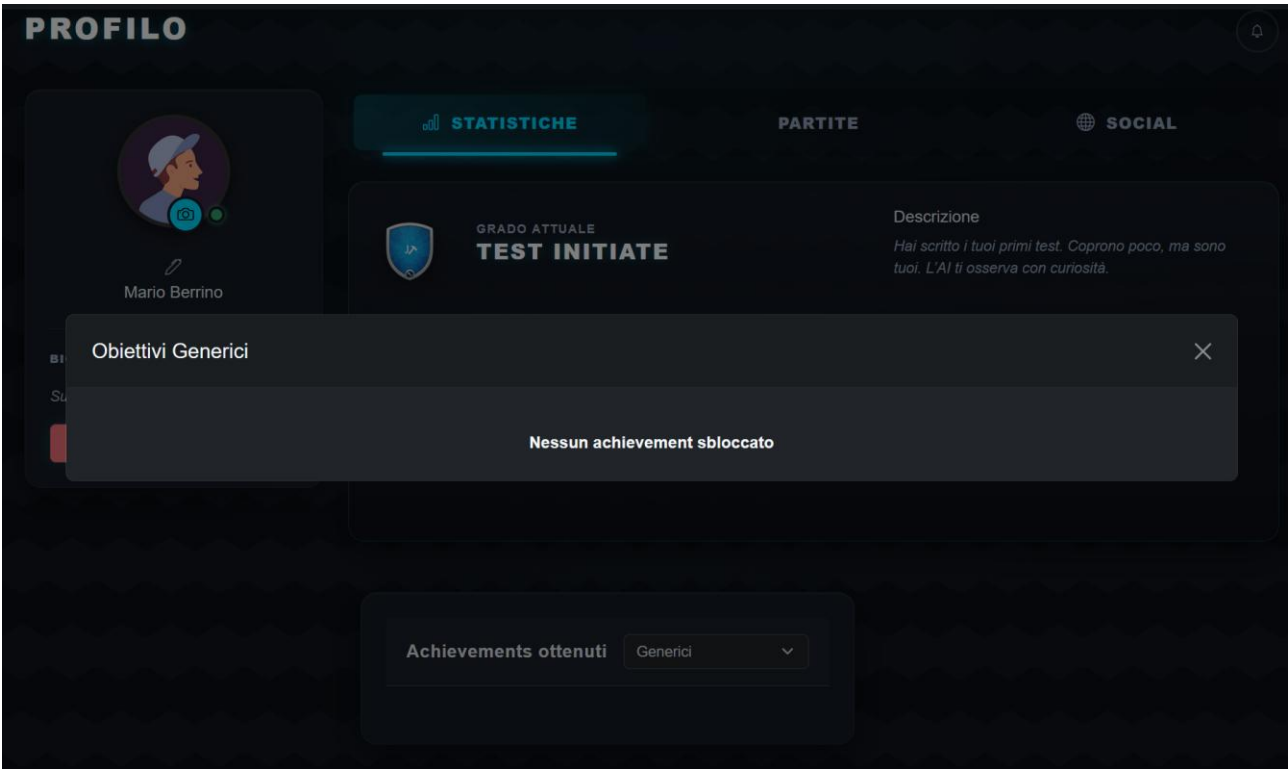


Figura 42 Visualizzazione messaggio di mancato achievements

	game_progress_id			achievements	
1		2		instructionsAndWeakMutation	
2		2		instructions	
3		3		allBeatenOneClass	
4		4		allBeatenTwoClass	

Figura 43 Achivements della singola partita

	player_progress_id			global_achievements	
1		1		firstMatchWon	
2		1		thirdMatchWon	

Figura 44 Achivements globali

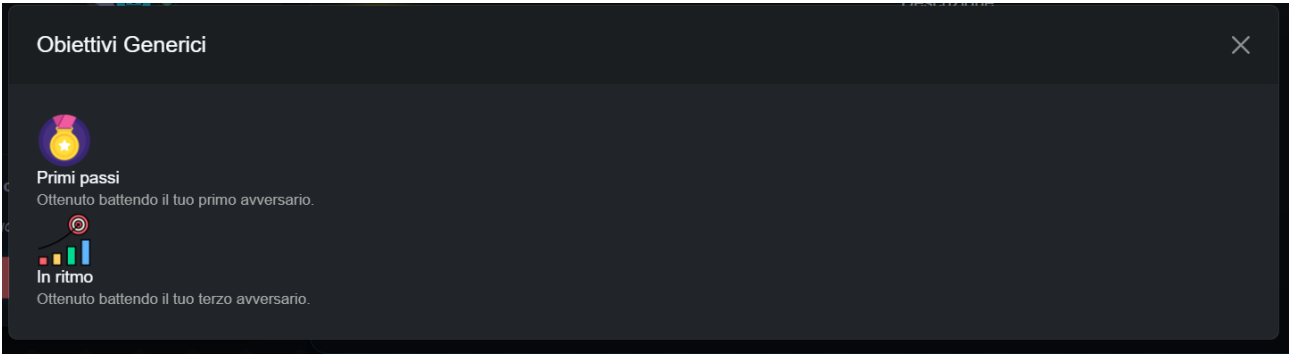


Figura 45 Vista obiettivi generici

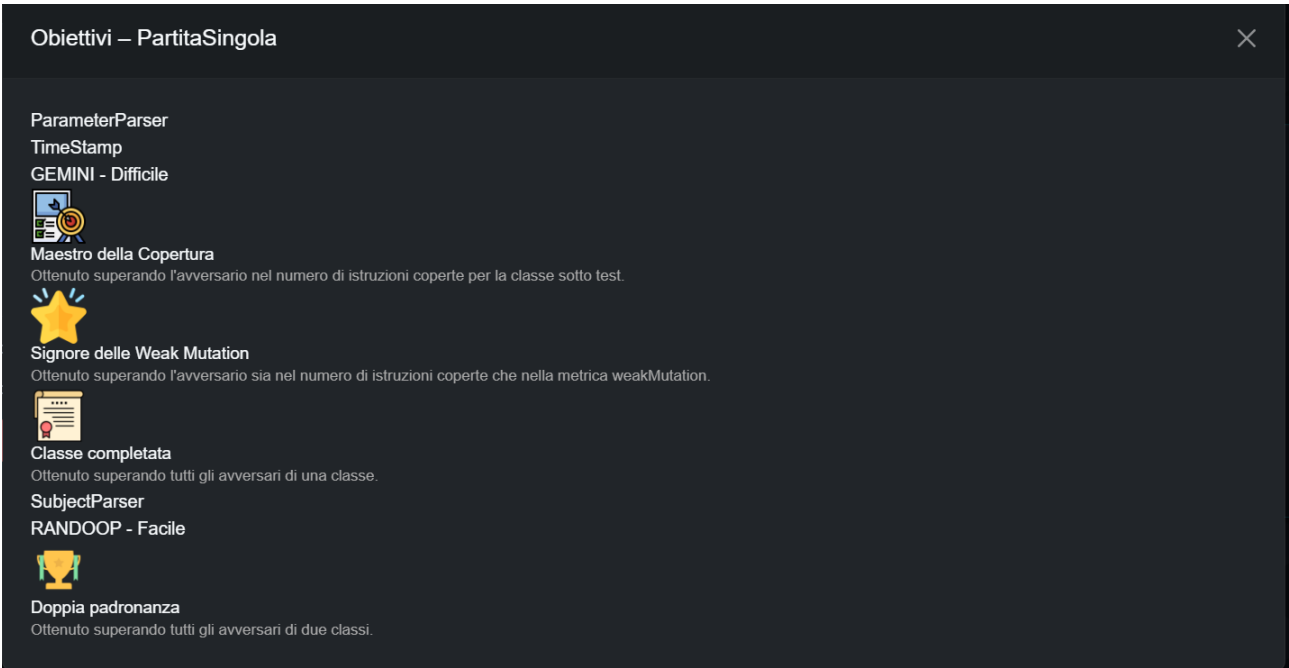


Figura 46 Vista degli obiettivi per partita singola

- **RF4 (gestione persistenza profilo)**

Il test ha previsto la modifica della biografia e del nickname tramite l'interfaccia dedicata. È stato confermato che, al click sul pulsante "Salva Modifiche", i dati vengano inviati correttamente al microservizio T23 e che rimangano persistenti anche dopo il ricaricamento della sessione. Inoltre è stato testato il limite di caratteri sia per biografia (max 200 caratteri) sia per Nickname(max 20 caratteri).

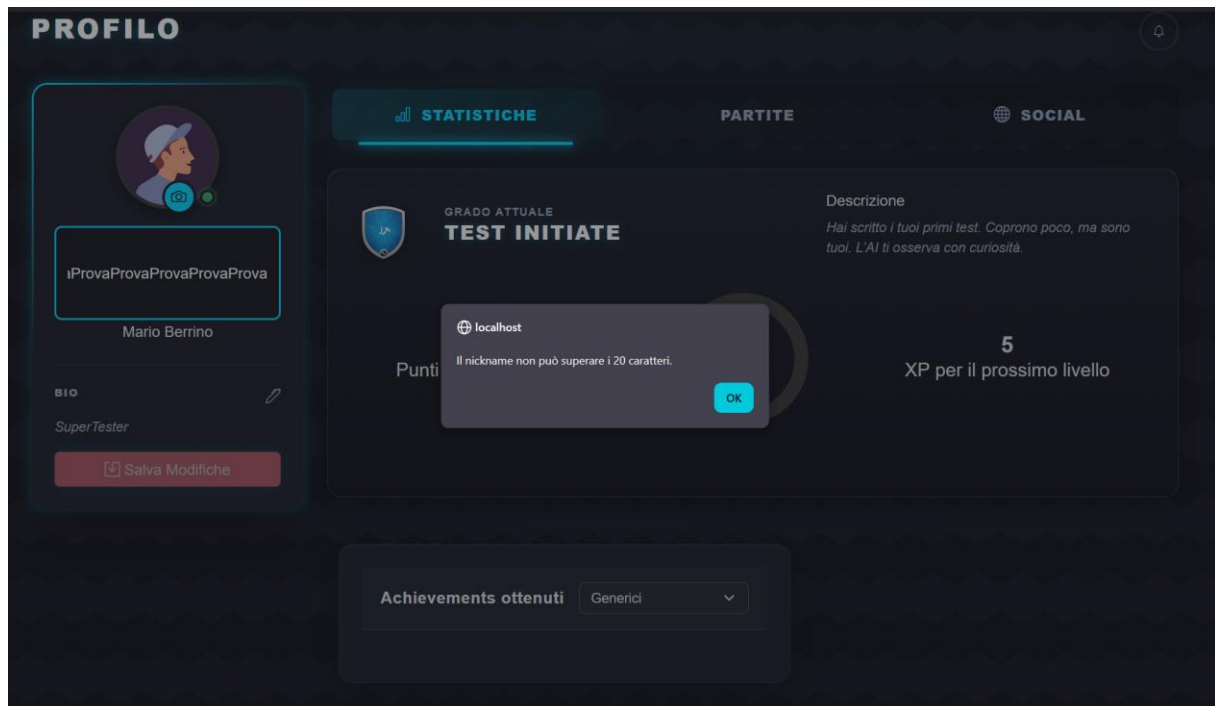


Figura 47 Avviso errore numero caratteri nickname

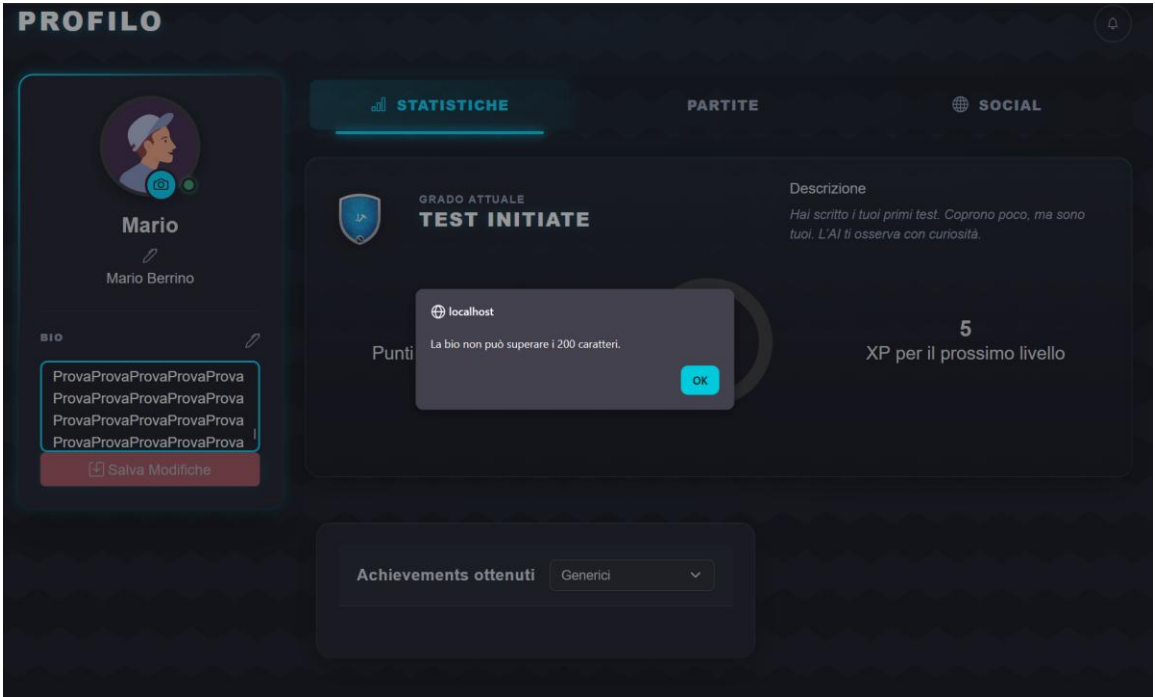


Figura 48 Avviso errore bio

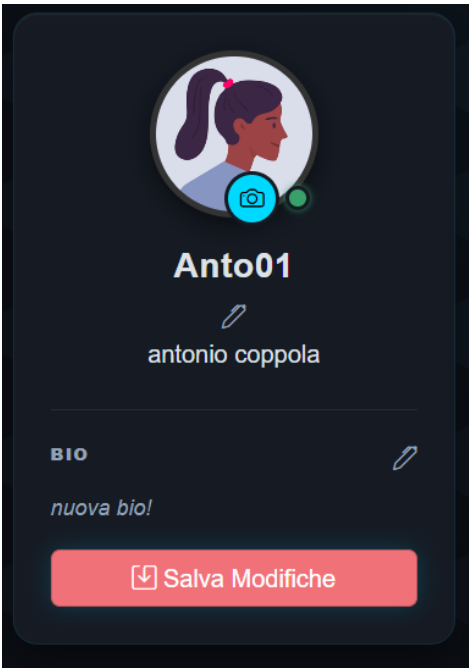


Figura 49 Vista profilo modificato

bio	email	name	nickname
nuova bio!	antonio.coppola34@studenti.un...	antonio	Anto01

Figura 50 Modifiche nel DB dopo aver effettuato il salvataggio delle modifiche

- **RF5 (navigazione modulare)**

È stata validata la navigazione tra le tab "STATISTICHE", "PARTITE" e "SOCIAL". Il test ha confermato che il passaggio avviene in modalità asincrona, aggiornando il modulo centrale senza ricaricare l'intera pagina web, garantendo fluidità all'esperienza d'uso.

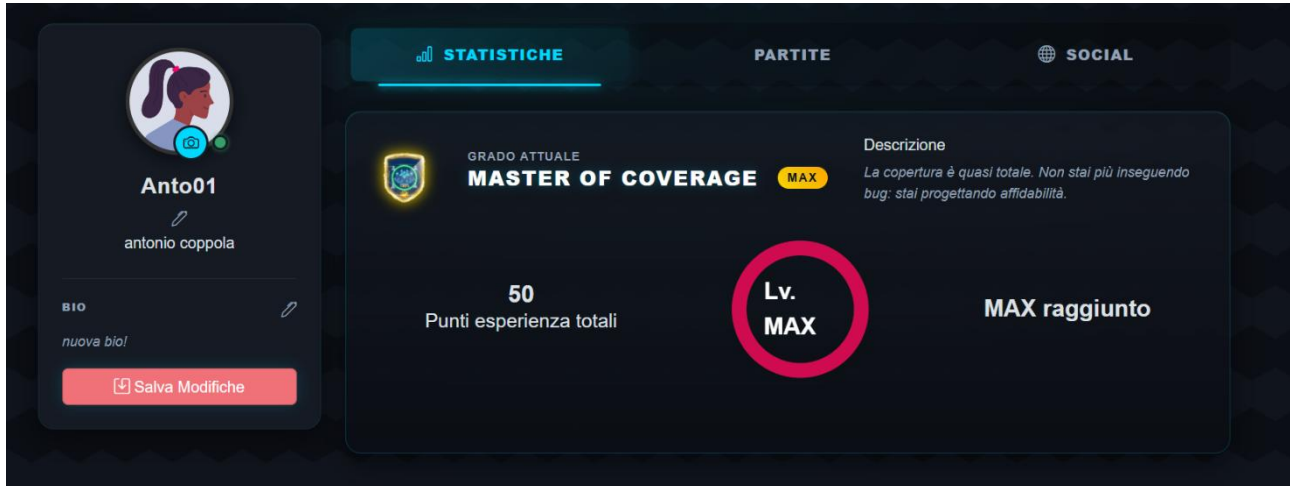


Figura 51 Verifica visualizzazione delle statistiche

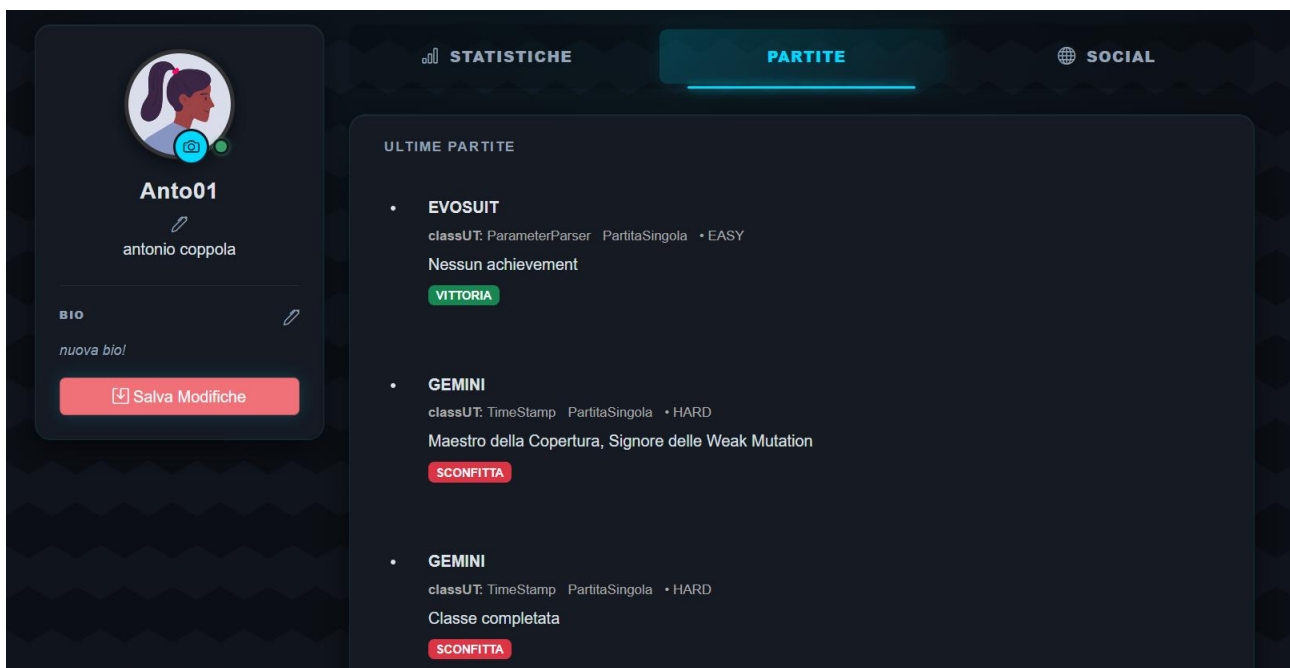


Figura 52 Verifica visualizzazione delle partite

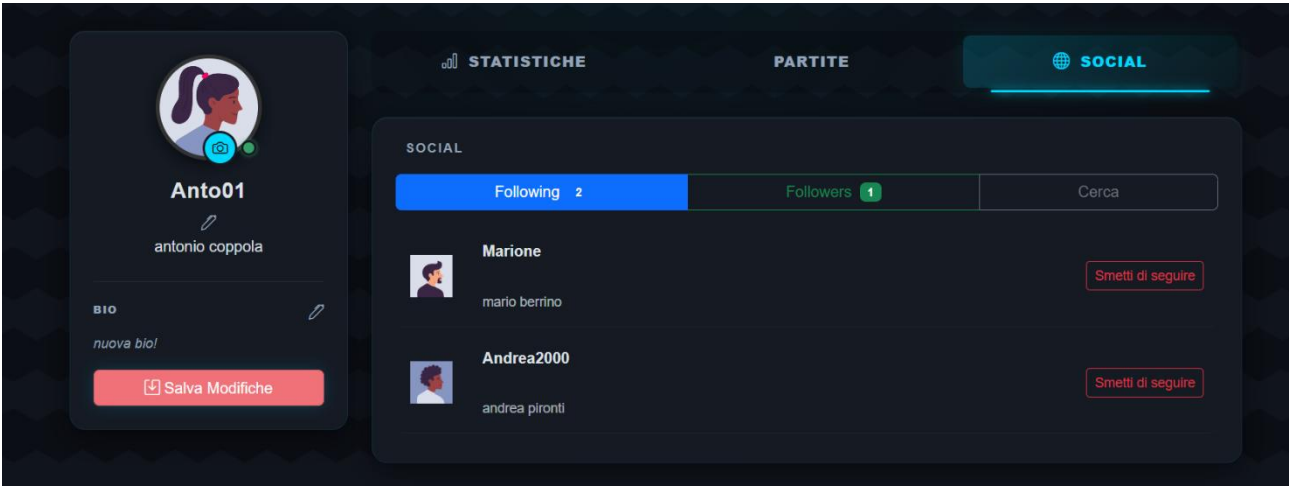


Figura 53 Verifica visualizzazione delle partite

- **RF6 e RF7 (progressi e storico partite)**

Si è verificata la corretta visualizzazione degli indicatori di livello (Lv. MAX), del rango ("Master of Coverage") e della barra XP, oltre alla consultazione dei dati storici relativi ai risultati e agli avversari nelle apposite sezioni. Inoltre sono stati testati i casi limite in cui si prova ad inserire punti XP maggiori, uguali o minori di quelli massimi.

WHERE	ORDER BY
id experience_points player_id	
1	2
1	70

Figura 54 Punti esperienza presenti nel database



GRADO ATTUALE

MASTER OF COVERAGE

MAX

Descrizione

La copertura è quasi totale. Non stai più inseguendo bug: stai progettando affidabilità.

70

Punti esperienza totali

Lv.

MAX

MAX raggiunto

Figura 55 Verifica nel caso di punti ottenuti superiori al massimo

WHERE	ORDER BY
id experience_points player_id	
1	2
1	34

Figura 56 Punti esperienza minori del punteggio massimo



Figura 57 Punti minori del massimo

id	experience_points	player_id
1	1	50
		2

Figura 58 Punteggio pari al punteggio massimo



Figura 59 Punti ottenuti pari al massimo

	id	is_winner	opponent_id	players_progresses_id
1	1	• true	1	1
2	2	false	3	1
3	3	false	3	1
4	4	• true	4	1

Figura 60 Partite disputate dai giocatori

ULTIME PARTITE

• RANDOOP

classUT: SubjectParser PartitaSingola • EASY

Doppia padronanza

VITTORIA

< Indietro

Avanti >

Figura 61 Storico delle partite

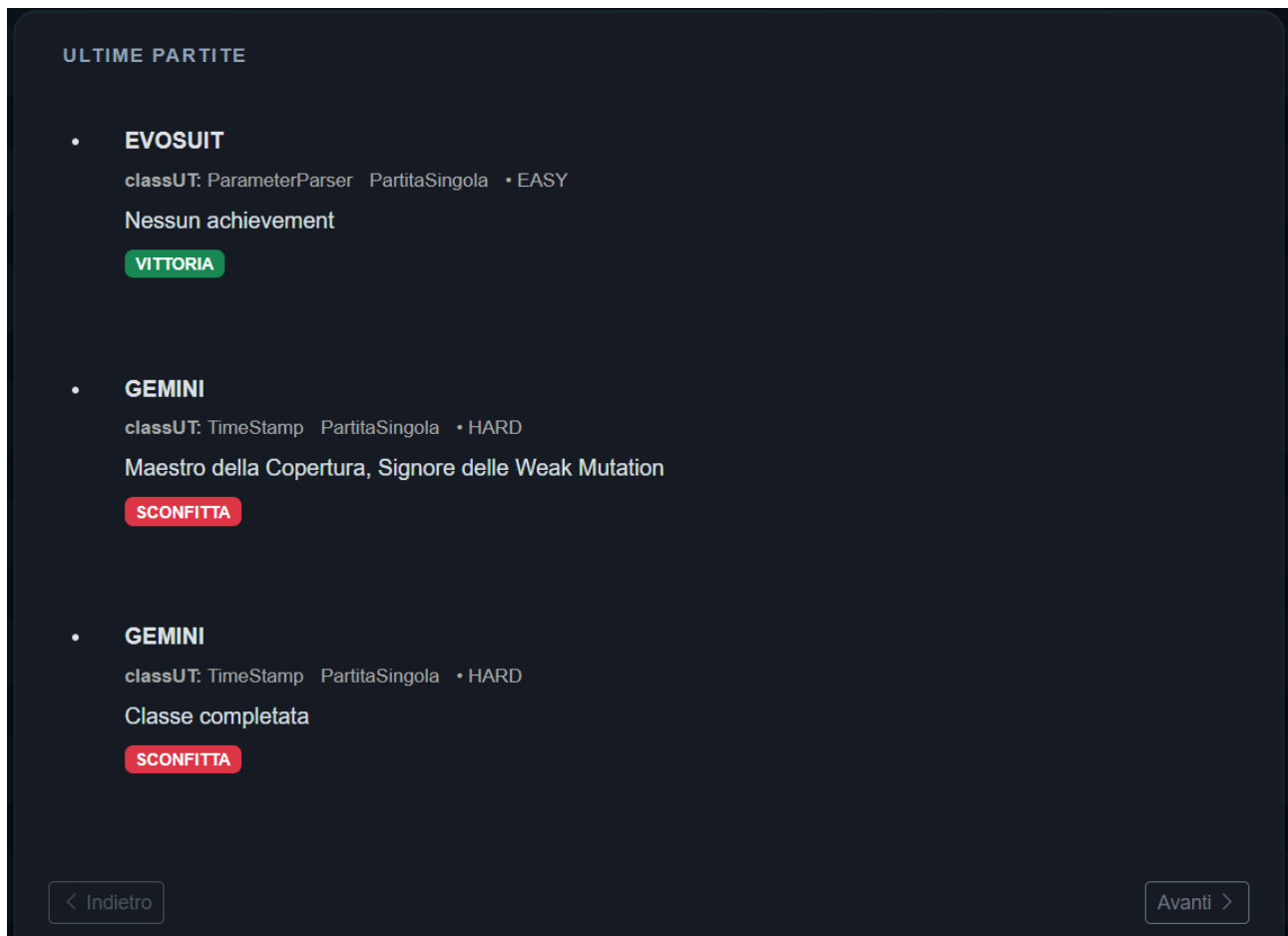


Figura 62 Visualizzazione delle partite giocate

- **RF8 (gestione social)**
Il test ha confermato la possibilità di gestire le relazioni (follower/following) e di ricercare altri giocatori, validando l'integrazione delle funzionalità social all'interno del profilo.

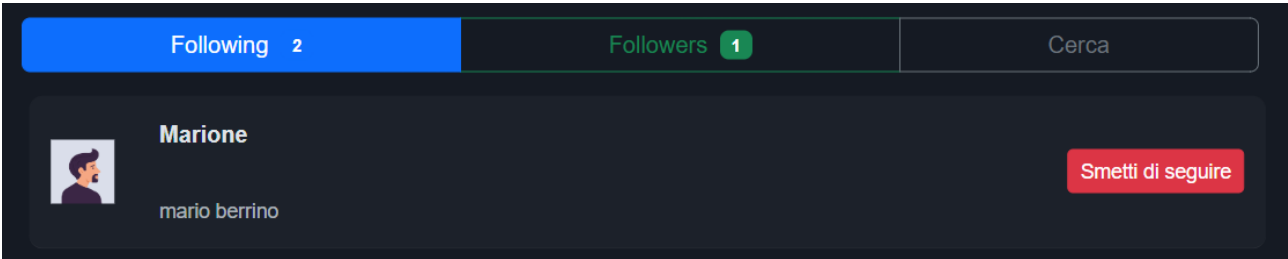


Figura 63 Vista iniziale dei seguiti

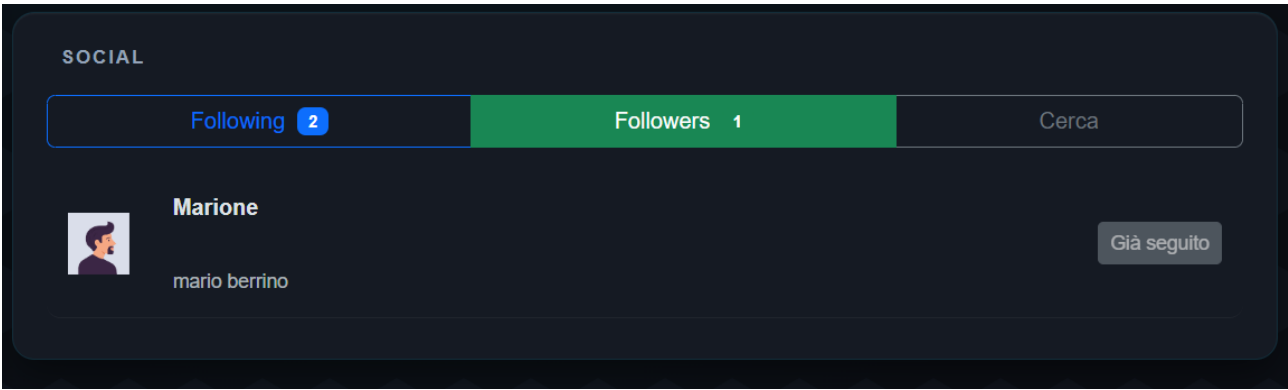


Figura 64 Vista iniziale dei Followers

WHERE		ORDER BY			
	follower	following	follower_id	following_id	
1	3	4	3	4	
2	3	5	3	5	
3	4	3	4	3	

Figura 65 Stato iniziale, giocatore "Marione" è seguito dall'utente

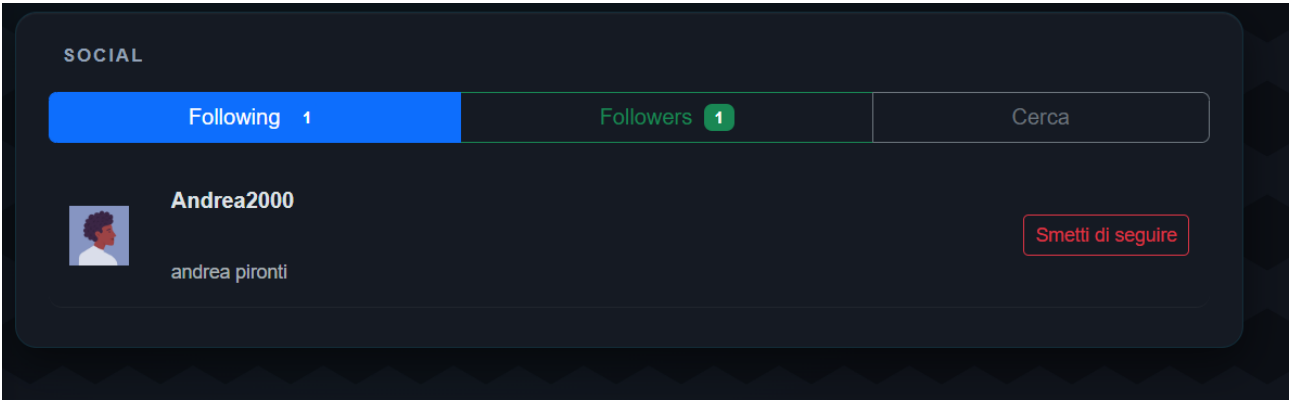


Figura 66 Rimozione del following “Marione”

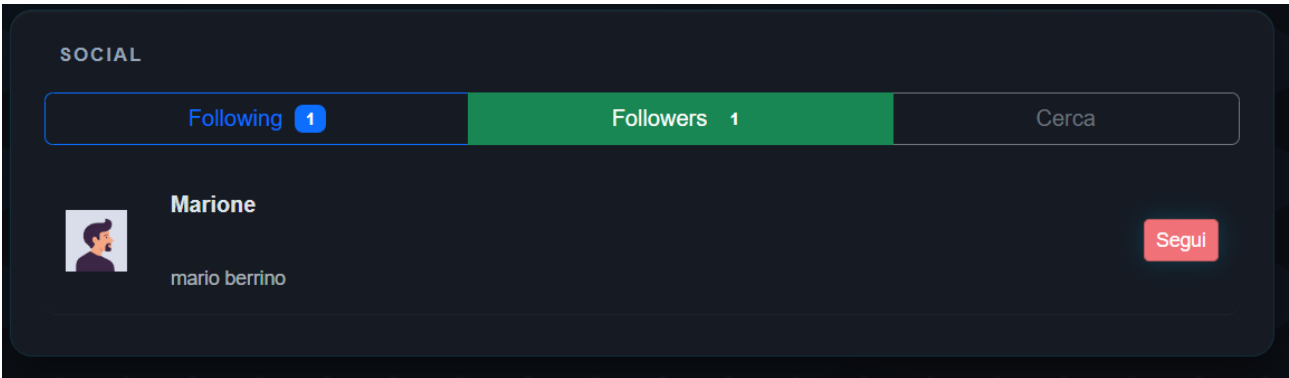


Figura 67 Vista dei follower successiva al “Smetti di seguire” riferito a “Marione”

	WHERE			ORDER BY	
	follower	following	follower_id	following_id	
1	3	5	3	5	
2	4	3	4	3	

Figura 68 Stato successivo al "Smetti di seguire" riferito a "Marione"

3.1.2 Test dei requisiti non funzionali:

Oltre alle funzionalità, è stata testata la qualità strutturale e l'usabilità del sistema:

- **RNF1 (estensibilità)**
L'architettura modulare è stata messa alla prova verificando che l'aggiunta di nuove features per future sezioni non richiedesse modifiche strutturali al core della pagina profilo (basta aggiungere una nuova sezione nel corpo della parte dinamica).
- **RNF2 (usabilità)**
L'interfaccia è stata testata per evitare il sovraffollamento informativo. La disposizione degli elementi (sidebar per dati fissi e corpo centrale per dati dinamici) è risultata intuitiva e i tempi di apprendimento per l'utente sono minimi.
- **RNF3 (coerenza)**
È stato eseguito un test di coerenza visiva confermando che gli elementi grafici, i font e i colori (es. il contrasto tra i toni scuri del background e i richiami cromatici accesi) siano uniformi tra la pagina profilo e la sezione achievement.
- **RNF4 (manutenibilità)**
È stata verificata la conformità dei modelli dati scambiati con il microservizio T23 rispetto alle convenzioni di sviluppo prestabilite, facilitando future operazioni di manutenzione o aggiornamento del codice.

L'attività di testing non ha evidenziato anomalie critiche, fatte le seguenti eccezioni:

- Il caricamento della lista giocatori risulta vincolato alla ricezione di un evento di input, indipendentemente dalla presenza di caratteri nel campo di ricerca.
- Si ha un problema nella visualizzazione delle partite quando l'ID dell'opponent coincide con l'ID del giocatore stesso.

A seguito della validazione si è arrivati alla conclusione che l'intervento sul profilo utente ha conseguito tutti i requisiti.

CAPITOLO 4 Sviluppi futuri

4.1 Evoluzioni future

In conclusione, l'analisi architeturale ha permesso di indentificare i seguenti margini di miglioramento:

- Assenza di un sistema di notifiche in tempo reale.
- Estensione della gestione dei contenuti multimediali per consentire l'upload di avatar definiti dall'utente.
- Mancanza di una funzionalità di chat tra utenti.

Riferimenti Bibliografici e Sitografia

Bibliografia

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Fondamentale se hai citato Pattern come MVC o Singleton).

Fowler, M. (2012). *Patterns of Enterprise Application Architecture*. Addison-Wesley. (Utile per la parte relativa all'architettura dei dati e dei microservizi).

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media. (Ideale dato che il progetto interagisce con il microservizio T23).

Uso delle dispense utilizzate durante il corso.

Sitografia

Oracle Documentation. *Java Platform, Standard Edition (Java SE) Documentation*. Disponibile su: <https://docs.oracle.com/en/java/>

Spring Framework. *Spring Boot Reference Documentation*. Disponibile su: <https://spring.io/projects/spring-boot>

JetBrains. *IntelliJ IDEA Documentation*. Disponibile su: <https://www.jetbrains.com/help/idea/>

Docker Documentation. *Docker Containerization Reference Guide*. Disponibile su: <https://docs.docker.com/>

Mozilla Developer Network (MDN). *JavaScript and Web Technologies Reference*. Disponibile su: <https://developer.mozilla.org/>